

Genomics Data Visualization

with Python · Day 3 · Production-
grade tools

Dr. Loukas Theodosiou // Day 3 // June 2026

2026-06-24

Welcome to Day 3

PRODUCTION-GRADE TOOLS • DAY 03

Why Reach for a Package?

THE BUILD-VS-BUY DECISION

Days 1-2 built every chart from matplotlib primitives. Today we use the libraries.

The Python primitives we used on days 1-2

(`ax.scatter`, `mpatches.Polygon`, `Rectangle`) **teach**

what every chart is made of — but for a paper figure, a polished publication-grade package is faster, more robust, and battle-tested.

Today, **one package per chart**:

- **PyComplexHeatmap** — clustered heatmaps with annotations
- **pyCircIz** — circular genome plots (cross-ref to day 2)
- **Toytree** — phylogenetic trees
- **pyMSAviz** — MSA viewers
- **DashBio** — interactive Plotly widgets
- **pyGenomeTracks** — config-driven genome tracks

- **dna_features_viewer** — gene / plasmid maps
- **JCVI / MCscan** — synteny and comparative
- **UpSetPlot** — set intersections
- **PyWaffle** — waffle compositions
- **gget** — gene + pathway lookups
- **Biopython** — Bio.Phylo, Bio.Graphics

THE TRADE-OFF

You know how a heatmap works from matplotlib. Now learn the **package that does it in 3 lines** — and how it falls down when you push it.

The Setup Cell

LOGISTICS · DAY 03

One install pulls every package today needs

```
1 pip install -q "git+https://github.com/loukesio/dataviz-genomicsdata.git@Python_2026"
```

```
1 # the theme: Economist palette + IBM Plex Mono rcParams, applied on import
2 from genomics_course.theme import *
3
4 # loaders bundled with the course package – no network needed
5 from genomics_course.data import (
6     load_deseq2, load_admixture, load_gwas,
7     load_timecourse, load_coexpression, load_expression,
8     load_expression_meta, load_microbiome, load_peaks,
9     load_peak_genes, load_qc, load_variant_classes, load_enrichment,
10 )
11
12 import numpy as np, pandas as pd
13 import matplotlib.pyplot as plt
14 import seaborn as sns
```

Each section below opens with its **own import line** for the package it teaches, so you can copy a single section into Colab and run it in isolation.

PyComplexHeatmap

CLUSTERED HEATMAPS WITH ANNOTATIONS

Install & Import

PYCOMPLEXHEATMAP • SETUP

One library that does dendrograms, annotation bars and legends in a single call

`PyComplexHeatmap` is a Python port of R's celebrated **ComplexHeatmap**. It is pandas-friendly: hand it a `DataFrame` and a few annotation `Series`, and it draws **row + column dendrograms, multiple annotation tracks**, and a tidy **legend** without you having to compose any of that by hand.

```
1 pip install PyComplexHeatmap
```

```
1 # the package exposes everything we need through one namespace alias
2 import PyComplexHeatmap as pch
3
4 # the two loaders we'll use today live in the course package
5 # (already imported in the deck-wide setup chunk: load_expression, load_expression_meta)
```

Compared with `sns.clustermap`, `PyComplexHeatmap` is the one to reach for **as soon as you need more than one annotation row** — the kind of figure that ends up in a Nature paper.

The Data

PYCOMPLEXHEATMAP · DATA PEEK

A 40-gene × 24-sample expression matrix, plus per-sample metadata

`load_expression()` returns the **log2 TPM matrix** —
genes on rows, samples on columns.

`load_expression_meta()` returns the **sample-level metadata** we will paint as annotation bars on top of the heatmap.

```
1 expr = load_expression()      # 40 genes × 24 samples (log2 TPM)
2 expr.iloc[:5, :6]            # peek 5 genes × 6 samples
```

	C01	C02	C03	C04	C05	
gene						
G000	8.056462	8.033121	7.388702	8.039300	8.680642	7.22
G001	8.009095	8.200648	8.748127	8.871357	7.636982	7.90
G002	8.308926	8.052406	7.713790	8.060053	7.555927	8.25
G003	6.592570	7.069818	7.127052	6.582166	7.239328	6.68
G004	7.512427	7.408371	7.465396	7.057753	8.094623	7.27

```
1 meta = load_expression_meta() # one row per sample
2 meta.head()
```

	sample	condition	batch	qc
0	C01	Control	A	0.950
1	C02	Control	B	0.893
2	C03	Control	C	0.910
3	C04	Control	A	0.920
4	C05	Control	B	0.986

Glossary: `sample` joins to the matrix columns ·
`condition` ∈ {Control, Drought} · `batch` ∈ {A, B, C} · `qc`
is a 0–1 quality score.

Step 1 • The Simplest Possible Heatmap

PYCOMPLEXHEATMAP • BUILD

Two booleans get us clustered rows and clustered columns for free

`ClusterMapPlotter` is the workhorse. Hand it the matrix, ask it to cluster both axes, pick a diverging colormap, and you already see the Control/Drought split as the two big column-dendrogram branches.

```
1 import PyComplexHeatmap as pch           # the package al
2
3 fig = plt.figure(figsize=(8, 5))          # CREATE the can
4 hm = pch.ClusterMapPlotter(              # CALL the workh
5     data=expr,                           # 40 × 24 matrix
6     row_cluster=True,                    # cluster genes
7     col_cluster=True,                    # cluster sample
8     cmap="RdBu_r",                       # diverging: blu
9     show_rownames=False, show_colnames=True, # 40 genes is tc
10 )
```

```
Starting..
Calculating row orders..
Reordering rows..
Calculating col orders..
Reordering cols..
Plotting matrix..
Collecting legends..
Plotting legends..
Estimated legend width: 7.5 mm
```

`row_cluster=True` runs hierarchical clustering on the **rows** and draws a dendrogram on the left;

`col_cluster=True` does the same for **columns** along the top. The order of rows/columns in the final picture follows those dendrograms.

Step 2 • A Column Annotation for Condition

PYCOMPLEXHEATMAP • BUILD

One coloured strip above the heatmap names the story

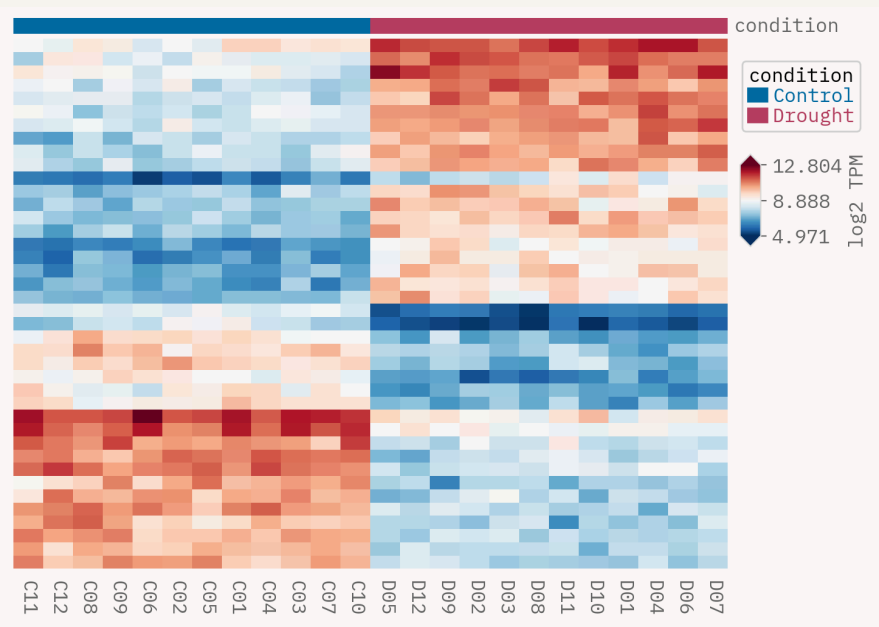
We build a `HeatmapAnnotation` for the **condition** column and pass it as `top_annotation`. The strip sits above the heatmap and lets the eye read “blue stripe = Control, red stripe = Drought” in a glance.

```
1 # align metadata to the matrix's column order so colours line up
2 meta = load_expression_meta().set_index("sample")
3 meta = meta.loc[expr.columns]                                # same order as
4
5 # one annotation track: condition, painted Control=BLUE / Drought=RED
6 col_ann = pch.HeatmapAnnotation(
7     condition=pch.anno_simple(                                # categorical s
8         meta["condition"],                                     # values to pai
9         colors={"Control": BLUE, "Drought": RED}),             # Economist pal
10    axis=1, verbose=0)                                         # axis=1 -> col
11
12 fig = plt.figure(figsize=(8, 5.4))                            # CREATE
13 hm = pch.ClusterMapPlotter(                                    # CALL
14     data=expr,
15     top_annotation=col_ann,                                    # the strip on
16     row_cluster=True, col_cluster=True,
17     cmap="RdBu_r",
18     show_rownames=False, show_colnames=True,
19     label="log2 TPM")
```

```
Starting..
Calculating row orders..
Reordering rows..
Calculating col orders..
Reordering cols..
Plotting matrix..
Collecting legends..
Plotting legends..
Estimated legend width: 27.765000000000004 mm
```

The `anno_simple` helper is the right tool for **categorical** tracks. For numeric tracks we'll switch to a different

helper in the next slide.



Step 3 • Stack More Annotation Tracks

PYCOMPLEXHEATMAP • BUILD

Categorical (batch) + numeric (qc) — two more rows of context

The header strip can carry as many tracks as you like.

We add a **discrete** batch track (A/B/C →

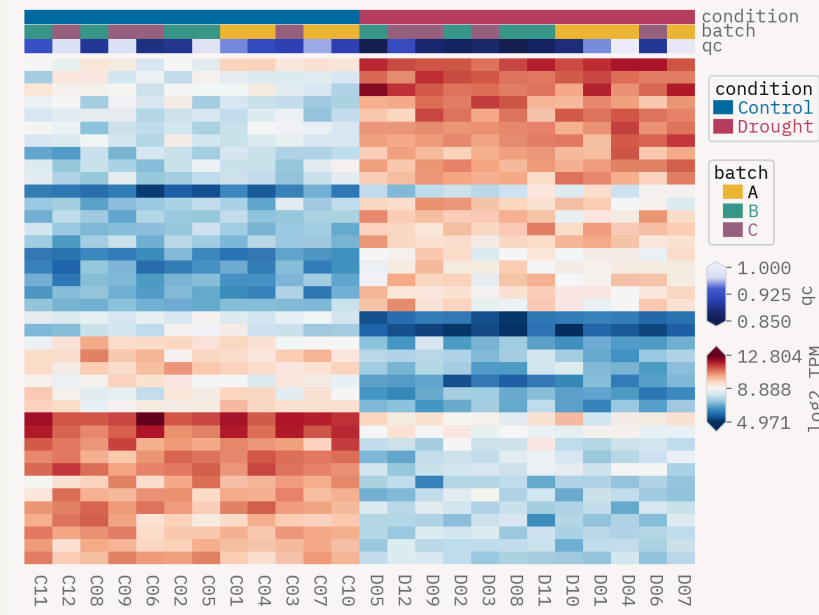
AMBER/GREEN/PURPLE) and a **continuous** qc track

on a sequential ramp. Each gets its own slice of the legend automatically.

```
1 # three annotation tracks at once, stacked top-to-bottom
2 col_ann = pch.HeatmapAnnotation(
3     condition=pch.anno_simple(                # row 1: categorical
4         meta["condition"],
5         colors={"Control": BLUE, "Drought": RED}),
6     batch=pch.anno_simple(                    # row 2: categorical
7         meta["batch"],
8         colors={"A": AMBER, "B": GREEN, "C": PURPLE}),
9     qc=pch.anno_simple(                       # row 3: numeric str:
10         meta["qc"],                          # values 0..1
11         cmap=econ_cmap("chicago")),         # sequential Econ
12     axis=1, verbose=0)
13
14 fig = plt.figure(figsize=(8, 5.8))           # CREATE
15 hm = pch.ClusterMapPlotter(                  # CALL
16     data=expr,
17     top_annotation=col_ann,                   # all three rows
18     row_cluster=True, col_cluster=True,
19     cmap="RdBu_r",
20     show_rownames=False, show_colnames=True,
21     label="log2 TPM")
```

```
WARNING: cmap is not a string!
Starting..
Calculating row orders..
Reordering rows..
Calculating col orders..
Reordering cols..
Plotting matrix..
Collecting legends..
Plotting legends..
Estimated legend width: 27.765000000000004 mm
```

Order matters: the first kwarg you pass becomes the **closest** strip to the heatmap. We put `condition` nearest because it carries the headline story.



Step 4 • Z-Score Rows, Polish the Frame

PYCOMPLEXHEATMAP • POLISH

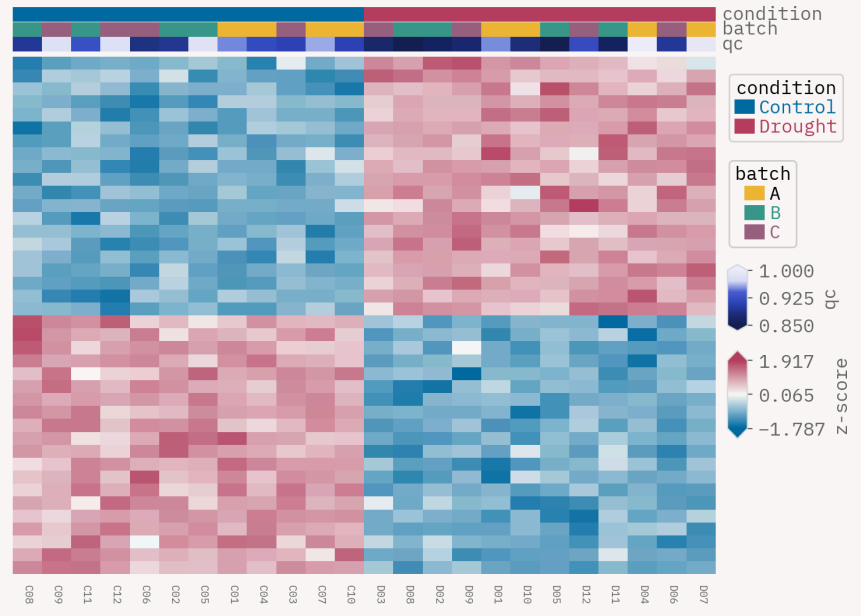
Raw log2 TPM hides the contrast — row-standardise first

We subtract each gene's mean and divide by its sd, so every row becomes “how many sds above/below this gene's own average”. Now the diverging cmap reads cleanly: **blue = below this gene's average, red = above**. Drought samples light up red, Control samples blue — the headline finding pops out.

```
1 # row-wise z-score: (x - row mean) / row sd
2 expr_z = expr.sub(expr.mean(axis=1), axis=0) \
3     .div(expr.std(axis=1), axis=0)          # standardised matrix
4
5 fig = plt.figure(figsize=(8.2, 5.8))      # CREATE
6 hm = pch.ClusterMapPlotter(               # CALL
7     data=expr_z,                          # z-scored values
8     top_annotation=col_ann,               # reuse from step
9     row_cluster=True, col_cluster=True,   # cluster both axes
10    cmap=econ_diverging,                   # Economist blue-red
11    show_rownames=False,                   # 40 genes -> labels
12    show_colnames=True,                    # keep sample IDs
13    col_names_side="bottom",               # under the heatmap
14    label="z-score",                       # legend title
15    xticklabels_kws={"labelsize": 7})      # smaller sample labels
```

```
WARNING: cmap is not a string!
Starting..
Calculating row orders..
Reordering rows..
Calculating col orders..
Reordering cols..
Plotting matrix..
Collecting legends..
Plotting legends..
Estimated legend width: 27.765000000000004 mm
```

A few editorial touches: drop the 40 row labels (they would overflow), put the sample IDs under the heatmap, and rename the legend to **z-score** so the reader knows the colour scale isn't raw expression.



PyComplexHeatmap • Takeaway

PYCOMPLEXHEATMAP • RECAP

One call. Clustering + annotation + legend.

THE RECIPE

1. **Build the matrix** (z-score the rows if you want a relative contrast). 2. **Build a** `HeatmapAnnotation` with one kwarg per track — `anno_simple` for categoricals, a `cmap` argument for numerics. 3. `ClusterMapPlotter(...)` with `row_cluster` / `col_cluster=True` and your annotation as `top_annotation` (or `left_annotation`). Reach for it the moment you have **more than one annotation row** or need a dendrogram. The matplotlib-from-primitives equivalent — `sns.clustermap` plus hand-built annotation bars + manual legend stitching — is fifty-plus lines for the same picture.

pyCirclize · Recap

CROSS-REFERENCE · ALREADY COVERED

What We Already Built

PYCIRCLIZE · DAY-2 RECAP

The five-step Circos walkthrough — done in day-2 _06

Day-2 _06_pycirclize.qmd — five-step build

1. **Sectors** — `{name: length}` defines the wedges.
2. **Axis + ticks** — `sector.axis()` and `sector.xticks_by_interval()`.
3. **Scatter track** — Manhattan-on-a-ring, `track.scatter(pos, -log10(p))`.
4. **Threshold ring** — dashed `track.line()` at `p = 5e-8`, hits redrawn red.
5. **Chord links** — `circos.link((sector_a, s, e), (sector_b, s, e))`.

Reach for pyCirclize when the story lives in *relationships across chromosomes* — translocations, Hi-C contacts, regulator-target pairs, microbial replicons — rather than within a single linear interval. Each ring trades per-chromosome zoom for cross-chromosome reach.

Three primitives carry the whole library:

- **Sectors** — wedges.
- **Tracks** — concentric data bands.
- **Links** — chords through the centre.

This section is a cross-reference only — go back to day-2 _06 for the full walkthrough.

One New Pattern • `Circos.chord_diagram`

PYCIRCLIZE • MATRIX-DRIVEN CHORDS

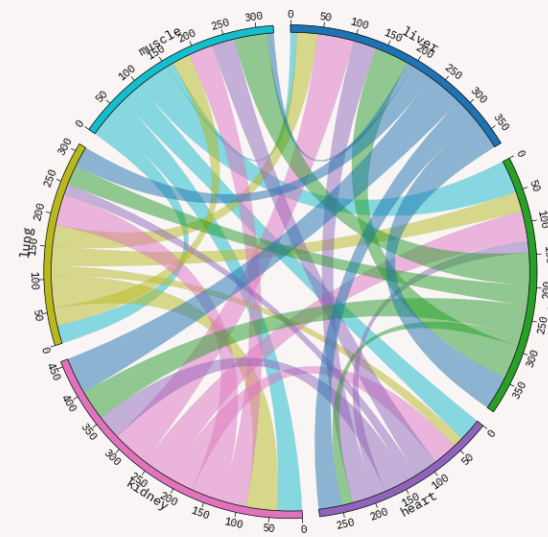
A symmetric matrix → a full chord diagram in one call

Day-2 built links one at a time. The classmethod `Circos.chord_diagram(matrix, ...)` takes the opposite path: hand it a **symmetric** `DataFrame` of pairwise weights (row label = sector, column label = sector, cell = chord thickness) and it auto-creates the sectors, draws every non-zero link, and scales each chord's width to the cell value. Perfect for tissue × tissue co-expression strengths, gene-family migration tables, or co-occurrence networks.

```

1 from pycirclize import Circos          # the same class
2
3 # six tissues – sector labels for both rows and columns
4 tissues = ["liver", "brain", "heart",
5            "kidney", "lung", "muscle"]
6
7 # synthesise a co-expression strength matrix:
8 # - rng integers from the day-3 seed (rng = default_rng(23))
9 # - symmetrise so chord A->B == chord B->A
10 # - zero the diagonal (no self-loops)
11 M = rng.integers(5, 80, size=(6, 6))    # raw asymmetric draws
12 M = (M + M.T) // 2                      # symmetric half-sum
13 np.fill_diagonal(M, 0)                  # drop self-links
14
15 # wrap as a labelled DataFrame – labels become sector names
16 mat = pd.DataFrame(M, index=tissues, columns=tissues)
17
18 # CREATE+CALL in one shot: chord_diagram is a classmethod that builds
19 # sectors from the row/col labels and links from every non-zero cell.
20 circos = Circos.chord_diagram(mat,
21                               space=4,    # 4deg gap between sectors
22                               cman="tab10" # one colour per sector

```



pyCirclize • Takeaway

PY CIRCLIZE • RECAP

Three primitives, two construction routes

THE RECIPE

1. **Sectors, tracks, links** — every Circos figure is some combination of these three. 2. **Manual build** (day-2 `_06`) — declare sectors, add tracks per sector, call `link()` for each chord. Full control; ideal when each link is its own story. 3. **Matrix build** (`Circos.chord_diagram`) — pass a symmetric DataFrame and let the library lay out sectors + draw every link. Ideal for co-expression / co-occurrence / migration tables where the matrix is the data.

Compare with: the linear Manhattan in day-2 `_04` (one chromosome at a time, no cross-chrom story) and the *JCVI synteny chord* coming up in `_08`, which is the same idea on two genomes — chords as homology between query and target rather than co-expression between tissues. Three names, one geometry.

Toytree

PHYLOGENETIC TREE DRAWING

Toytree • Install + Theory

TOYTREE • SETUP

A fluent tree API that returns a Toyplot canvas

```
1 pip install toytree
```

```
1 import toytree           # the package itself
2 import toyplot           # the rendering engine Toytree draws onto
3 import toyplot.svg, toyplot.png # explicit backends for export
```

Theory. Toytree wraps an `ete3`-style tree object behind a single fluent `.draw()` call. It reads **Newick / Nexus** strings or files, lets you style **per-tip / per-node / per-edge** with simple list arguments, and ships a collection of layouts — rectangular, circular (“c”), unrooted. Under the hood it does **not** use matplotlib; it renders to a **Toyplot Canvas** which is SVG-native and embeds cleanly in HTML decks like this one. For a matplotlib- native option see `Bio.Phylo.draw` in section 12.

Toytree • The Data

TOYTREE • DATA PEEK

Hand-write a Newick string of 9 taxa

A Newick string nests sister taxa in parentheses, with `:branch_length` after each name. Below is a small balanced clade of 9 species (A ... I). We feed it to `toytree.tree()` and peek at the parsed tree.

```
1 import toytree                                # tree object + .draw API
2
3 # 9-tip balanced phylogeny; numbers after ":" are branch lengths
4 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0," \
5          "(E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);"
6
7 tre = toytree.tree(newick)                    # parse Newick -> ToyTree
8 print("ntips:", tre.ntips)                   # how many leaves
9 print("tips :", tre.get_tip_labels())        # leaf names in plot order
```

```
ntips: 9
tips : ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']
```

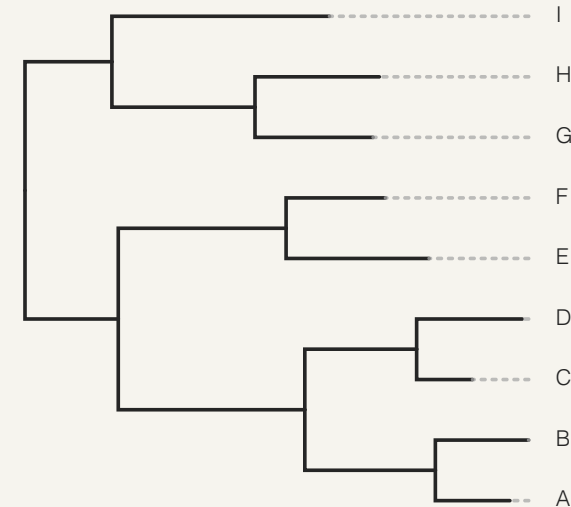

Toytree · Step 1 — Simplest Draw

TOYTREE · BUILD

1. `.draw()` returns (canvas, axes, mark)

The minimum call. `.draw()` returns a 3-tuple: a **Toyplot Canvas** (the SVG container), an **axes** (the coordinate frame), and a **mark** (the tree graphic itself). The canvas embeds straight into Quarto reveal.js as HTML/SVG when it is the chunk's last expression — no matplotlib involved.

```
1 import toytree                # the package
2
3 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0, \
4          (E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);"
5 tre = toytree.tree(newick)    # parse Newick once
6
7 # CREATE / CALL combined — .draw() builds canvas + axes + tree mark
8 canvas, axes, mark = tre.draw() # returns 3-tuple
9     width=420, height=380,      # canvas size in px
10     tip_labels_align=True,      # right-align leaf l
11 )
12
13 # Quarto auto-renders the Toyplot Canvas to inline SVG;
14 # fallback: toyplot.svg.render(canvas, "tre.svg") then 
15 canvas                        # last expression ->
```



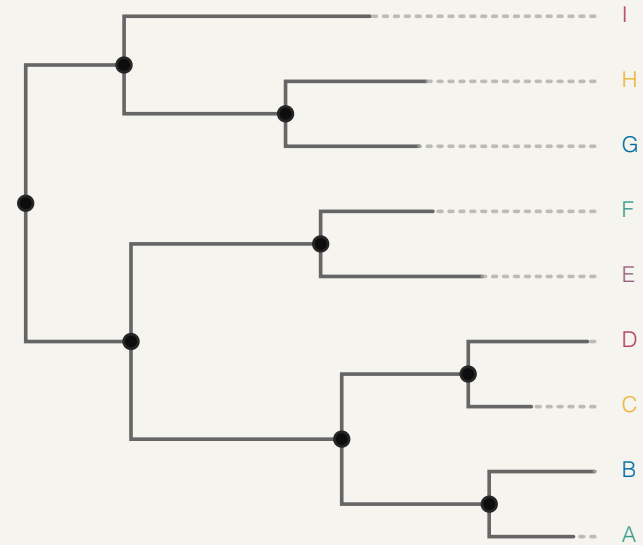
Toytree · Step 2 — Style Tips and Nodes

TOYTREE · BUILD

2 · Per-tip colours, larger nodes at the root

`tip_labels_colors` takes one colour per tip (in `get_tip_labels()` order). Pass a list straight from the Economist palette. `node_sizes` accepts a scalar or one size per node — give the deeper nodes more weight so the eye finds the backbone.

```
1 import toytree          # the package
2 from itertools import cycle          # pad palette to 1
3
4 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0," \
5         "(E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);"
6 tre = toytree.tree(newick)          # parse once
7
8 # one colour per tip from the Economist 5-hue qualitative palette
9 ECON_QUAL = [GREEN, BLUE, AMBER, RED, PURPLE]          # course palette
10 tip_colors = [c for c, _ in zip(cycle(ECON_QUAL),          # cycle palette
11                                tre.get_tip_labels())] # to length ntips
12
13 # CREATE / CALL — draw with per-tip colour + variable node sizes
14 canvas, axes, mark = tre.draw(
15     width=460, height=400,          # canvas size
16     tip_labels_align=True,          # cleaner leaf col
17     tip_labels_colors=tip_colors,    # one colour per 1
18     node_sizes=8,                  # base size for ev
19     node_colors=INK,               # nodes in ink-gra
20     edge_colors=MUTED,             # edges in muted
21 )
22 canvas
```



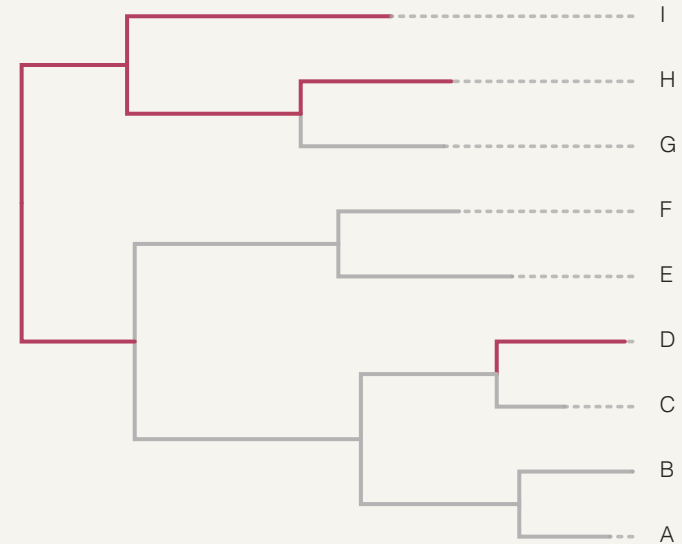
Toytree · Step 3 — Colour a Clade

TOYTREE · BUILD

3 · Spotlight the (A,B,C,D) clade in RED

Find the **most-recent common ancestor (MRCA)** of a set of tips with `get_mrca_node()`, list its descendants, then build per-edge colours: RED on the clade's edges, GREY for everything else. The story-colour rule applies — one clade in red, all else neutral.

```
1 import toytree          # the package
2
3 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0," \
4         "(E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);"
5 tre = toytree.tree(newick)      # parse once
6
7 # pick the clade we want to spotlight by its leaf set
8 clade_tips = ["A", "B", "C", "D"]      # the four sister
9 mrca = tre.get_mrca_node(*clade_tips)  # MRCA node object
10 clade_idx = {n.idx for n in mrca.get_descendants()} | {mrca.idx} # node :
11
12 # build edge colours — RED for clade edges, GREY for the rest
13 edge_colors = [RED if n.idx in clade_idx else GREY      # one colour per
14               for n in tre.treenode.traverse()]
15
16 # CREATE / CALL — draw with the highlighted edges
17 canvas, axes, mark = tre.draw(
18     width=480, height=400,          # canvas size
19     tip_labels_align=True,          # leaf column ali
20     edge_colors=edge_colors,        # per-edge colour
21     edge_widths=2.2,               # thicker to read
22     node_sizes=0,                  # nodes off-clade
```



Toytree • Step 4 — Circular Layout

TOYTREE • BUILD

4 • Same tree, radial display

Add `layout="c"` and the same data renders as a circular phylogeny — useful when you have many tips or want to compare to a circos-style figure. The spotlight colours port over unchanged; only the layout argument is new.

```
1 import toytree                                # the package
2
3 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0, \
4         \"(E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);\"
5 tre = toytree.tree(newick)
6
7 clade_tips = [\"A\", \"B\", \"C\", \"D\"]          # same clade
8 mrca = tre.get_mrca_node(*clade_tips)         # MRCA node
9 clade_idx = {n.idx for n in mrca.get_descendants()} | {mrca.idx}
10 edge_colors = [RED if n.idx in clade_idx else GREY    # same per-edge co
11                for n in tre.treenode.traverse() if not n.is_root()]
12
13 # CREATE / CALL — only layout="c" changes
14 canvas, axes, mark = tre.draw(
15     width=460, height=460,                        # square canvas fo
16     layout="c",                                    # "c" = circular
17     edge_colors=edge_colors,                       # spotlight ports
18     edge_widths=2.0,
19     tip_labels_align=True,                         # radial tip align
20 )
21 canvas
```

<toyplot.canvas.Canvas at 0x118d3d1c0>

Toytree • Takeaway

TOYTREE • RECAP

Five lines, publication-quality

THE RECIPE

1. `toytree.tree(newick)` parses every standard tree format.
2. `.draw()` returns a Toyplot Canvas — SVG-native, embeds straight into reveal.js / HTML / notebooks. No matplotlib needed.
3. **Per-tip / per-node / per-edge styling** via simple list arguments — `tip_labels_colors`, `node_sizes`, `edge_colors`.
4. **MRCA + descendants** is the standard pattern for clade highlights.
5. `layout="c"` flips rectangular to circular without changing anything else.

Compare with: `Bio.Phylo.draw` (section 12) gives you a matplotlib-native tree on the same `plt.subplots()` axes you use everywhere else — pick that one when you need to stack a tree above a heatmap. Reach for **Toytree** when the tree itself is the figure and you want pixel-perfect SVG output.

pyMSAviz

PRODUCTION MSA VIEWER

pyMSAviz · Install + Theory

PYMSAVIZ · SETUP

Five lines for a Clustal-style alignment figure

```
1 pip install pyMSAviz
```

```
1 from pymsaviz import MsaViz      # the one class that does everything
```

Theory. `pyMSAviz` reads **FASTA / Clustal / PHYLIP / Stockholm** alignment files (or a Biopython `MultipleSeqAlignment`), applies a **standard colour scheme** (`Clustal`, `Zappo`, `Taylor`, `Nucleotide`, ...), draws a **consensus track** at the bottom, prints **per-column letter counts**, and **wraps** long alignments across multiple rows. The output is a matplotlib `Figure`, so the cream-canvas `rcParams` and `IBM Plex Mono` apply automatically. Day-2 `_03` taught the primitives; this is the same picture, in five lines.

pyMSAviz · The Data

PYMSAVIZ · DATA PEEK

Synthesise 8 DNA sequences × 80 cols, write FASTA

`MsaViz` expects a file path or a biopython object. We build a small toy alignment in numpy (same approach as day-2 `_03`), then dump it to a temporary FASTA file. Print the first few sequences as a sanity check.

```
1 from pathlib import Path                # for temp file path
2 import tempfile                        # cross-platform temp dir
3
4 # build a small DNA alignment: 8 seqs x 80 cols, ~5% mutation, one indel
5 n_rows, n_cols = 8, 80                # alignment shape
6 bases = np.array(list("ACGT"))        # alphabet
7 base = rng.choice(bases, size=n_cols) # ancestor sequence
8 aln = np.tile(base, (n_rows, 1))      # 8 copies of ancestor
9 mutate = rng.random((n_rows, n_cols)) < 0.05 # 5% substitution mask
10 mutate[0] = False                     # keep seq0 as ancestor
11 aln[mutate] = rng.choice(bases, size=mutate.sum()) # apply substitutions
12 aln[5, 30:36] = "-"                  # 6-col deletion in seq5
13
14 # write to a temp FASTA file so MsaViz can read it
15 fasta_path = Path(tempfile.gettempdir()) / "toy_msa.fasta" # OS-temp FASTA path
16 with open(fasta_path, "w") as fh:    # plain FASTA writer
17     for i, row in enumerate(aln):    # one record per row
18         fh.write(f">seq{i}\n" + "".join(row) + "\n") # header + sequence
19
20 print("wrote:", fasta_path)           # confirm file location
21 print("first 2 sequences:")
22 for i in range(2):
```

```
wrote: /var/folders/fc/kqr5vp3xlwg4z_8jvxpwpqlm0000gn/T/toy_msa.fasta
first 2 sequences:
>seq0 AGAGCACGCGTTATCAATTCAGCTATGTAGATTCAAGAGCTGTCCGCACTTCTTAGATAC ...
>seq1 AGATCACGCGTTCTCAATTCAGTTATGTAGATTCAAGAGCTGTCCGCACTTCTTAGATAC ...
```

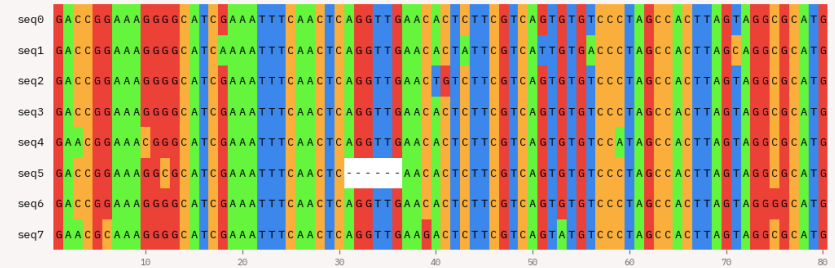

pyMSAviz · Step 1 — Simplest Viz

PYMSAVIZ · BUILD

1. `MsaViz(fasta).plotfig()` returns a matplotlib Figure

`MsaViz(path)` parses the alignment, picks a default colour scheme, and lays out the figure. `.plotfig()` returns the **matplotlib Figure** — the cream-canvas `rcParams` set in the theme apply automatically. Two lines.

```
1 from pymsaviz import MsaViz           # the one class
2
3 # fasta_path was written in the previous slide
4 mv = MsaViz(str(fasta_path))           # CREATE — parse + 1
5
6 # CALL — render to a matplotlib Figure (cream rcParams apply)
7 fig = mv.plotfig()                     # returns plt.Figure
8 fig.set_size_inches(9.5, 3.0)          # fit our two-column
9 plt.show()                             # render in the slide
```



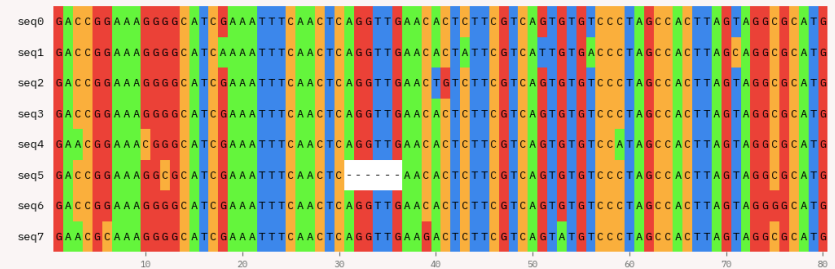
pyMSAviz · Step 2 — Colour Scheme

PYMSAVIZ · BUILD

2 · `color_scheme="Nucleotide"` for DNA

The default scheme is Clustal (protein-tuned). For DNA, switch to `color_scheme="Nucleotide"` — A/C/G/T get distinct hues and gaps stay neutral. For protein alignments use `"Clustal"`, `"Zappo"`, or `"Taylor"`. The scheme choice is the single most visible knob in the whole library.

```
1 from pymsaviz import MsaViz                # the one class
2
3 # only one new keyword vs step 1 — explicit colour scheme for DNA
4 mv = MsaViz(
5     str(fasta_path),                        # the same toy align
6     color_scheme="Nucleotide",              # A/C/G/T-specific c
7 )
8
9 fig = mv.plotfig()                          # CALL — render Figu
10 fig.set_size_inches(9.5, 3.0)              # match our column w
11 plt.show()
```



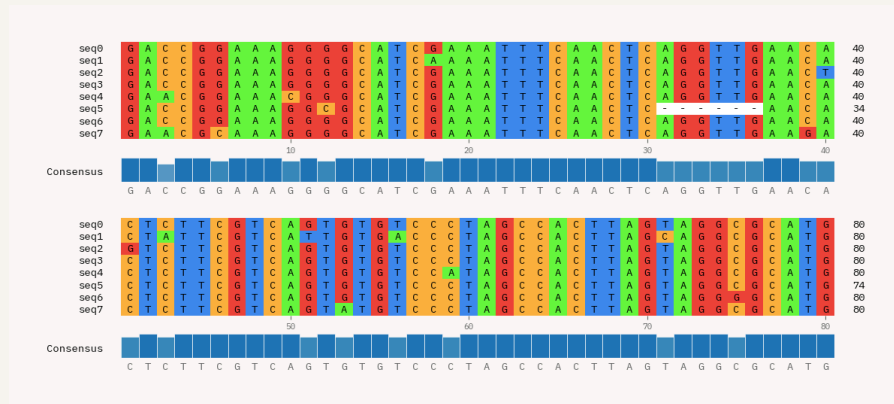
pyMSAviz · Step 3 — Consensus + Counts + Wrap

PYMSAVIZ · BUILD

3 · The canonical Clustal viewer look

Three keyword arguments turn the bare grid into the full Clustal-viewer layout: `show_consensus=True` adds the consensus row, `show_count=True` shows per-column letter counts on the right axis, and `wrap_length=40` breaks the alignment across multiple rows so 80 columns become two 40-wide stripes.

```
1 from pymsaviz import MsaViz          # the one class
2
3 # add three knobs: wrap, consensus row, per-column counts
4 mv = MsaViz(
5     str(fasta_path),                # same toy alignment
6     color_scheme="Nucleotide",      # DNA colours
7     wrap_length=40,                 # wrap every 40 cols
8     show_consensus=True,            # bottom consensus r
9     show_count=True,                # column number tick
10 )
11
12 fig = mv.plotfig()                  # CALL — render Fig
13 fig.set_size_inches(9.5, 4.2)      # taller — wrapped l
14 plt.show()
```



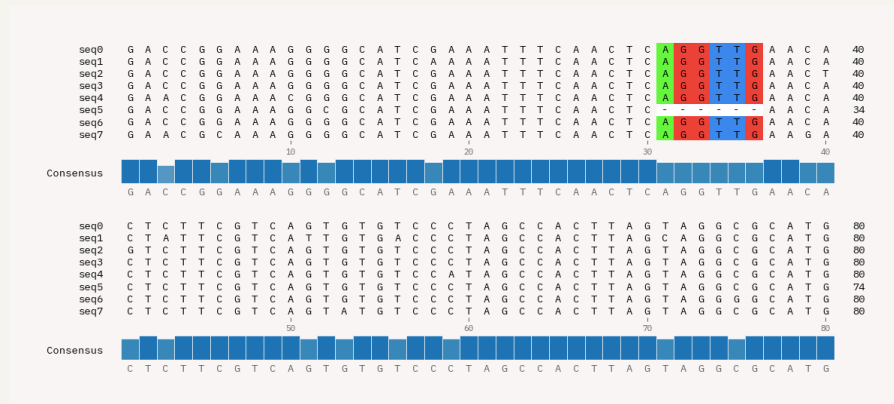
pyMSAviz · Step 4 — Highlight a Region

PYMSAVIZ · BUILD

4 · `set_highlight_pos([(start, end)])` to spotlight a motif

`set_highlight_pos` takes a list of `(start_col, end_col)` tuples (1-indexed, inclusive) and draws a box around each region. Use it to mark a motif, an indel block, a primer site, anything you want the reader to look at first. We spotlight the deletion block at columns 31–36.

```
1 from pymsaviz import MsaViz          # the one class
2
3 # rebuild with the same wrapped, consensus look from step 3
4 mv = MsaViz(
5     str(fasta_path),                  # same alignment file
6     color_scheme="Nucleotide",        # DNA colours
7     wrap_length=40,                   # two rows of 40 col
8     show_consensus=True,              # bottom consensus
9     show_count=True,                  # column counts axis
10 )
11
12 # box the indel region — columns 31..36 (1-indexed, inclusive)
13 mv.set_highlight_pos([(31, 36)])     # one or many (start
14
15 fig = mv.plotfig()                   # CALL — render Fig
16 fig.set_size_inches(9.5, 4.2)       # match step 3 height
17 plt.show()
```



pyMSAviz · Takeaway

PYMSAVIZ · RECAP

When an MSA needs to look like an MSA viewer

THE RECIPE

1. `MsaViz(fasta)` parses any FASTA / Clustal / PHYLIP / Stockholm file.
2. `color_scheme=` picks Clustal / Zappo / Taylor (proteins) or Nucleotide (DNA).
3. `wrap_length=`, `show_consensus=`, `show_count=` add the canonical Clustal-viewer scaffolding in three keyword arguments.
4. `set_highlight_pos([(s,e)])` boxes the regions the audience must look at.
5. `.plotfig()` returns a `matplotlib Figure` — cream-canvas rcParams and IBM Plex Mono apply for free.

Compare with: the day-2 `_03_msa` build teaches the **primitives** — letter grid, `imshow` colour grid, conservation bar, zoom — so you understand what every pixel is. Reach for **pyMSAviz** when the alignment **is** the figure and you want the production output without rebuilding the wheel.

DashBio

INTERACTIVE PLOTLY WIDGETS FOR GENOMICS

Install & Import

DASHBIO • SETUP

Plotly figure-generators built for bioinformatics — no Dash server needed

`dash-bio` is Plotly's bioinformatics widget collection. It ships inside Dash apps, but every widget is **also** a standalone *figure generator*: each call returns a regular `plotly.graph_objects.Figure`, which renders inline in Quarto / Jupyter exactly like any other Plotly chart. Hover for SNP IDs, scroll-zoom into peaks, no server required.

```
1 pip install dash-bio
```

```
1 # the canonical alias in the docs and tutorials
2 import dash_bio as dashbio
3
4 # we'll reuse loaders from the deck setup chunk:
5 #   load_gwas("height")      -> Manhattan widget
6 #   load_deseq2("drought")  -> Volcano widget
7 #   load_expression()       -> Clustergram widget
```

Reach for `dash-bio` when you want **interactivity** (hover tooltips, zoom, pan) and you'd rather assemble the chart in one line than rebuild the day-2 matplotlib version. For static, paper-ready figures the day-2 matplotlib sections still win on control.

The Catalogue

DASHBIO • CATALOGUE

Six widgets we'll touch — each one parallels a day-1 or day-2 chart

The widgets

- **ManhattanPlot** — interactive Manhattan with hover-for-SNP. Parallels day-2 `_04_manhattan.qmd`.
- **VolcanoPlot** — interactive volcano with effect-size + significance cutoffs. Parallels day-1 `_07_volcano.qmd`.
- **Clustergram** — heatmap + row/column dendrograms (Plotly's `sns.clustermap`). Parallels day-3 `_01_pycomplexheatmap.qmd`.
- **Circos** — link diagram on a circular ideogram. Parallels day-2 `_06_pycircize.qmd`.
- **AlignmentChart** — multiple-sequence alignment viewer. Parallels day-3 `_04_pymsaviz.qmd`.
- **NeedlePlot** — lollipops for variant effects along a protein. Parallels day-1 lollipop variants.

The pattern is the same for every widget: hand it a dataframe (or string), name the columns / kwargs, and the call returns a Plotly `Figure`. Whatever you can do with a Plotly figure — `update_layout`, `update_traces`, `to_html` — works on the DashBio output.

ManhattanPlot · One-Line Build

DASHBIO · MANHATTAN

Hand it the GWAS table, name the four columns, done

DashBio's `ManhattanPlot` takes the same GWAS table we built up by hand in day-2 (`chrom`, `pos`, `pval`, `snp`).

The horizontal threshold lines and the two-tone chromosome colours are drawn **for you**; hover tooltips expose every SNP's rsID.

```
1 import dash_bio as dashbio          # the package alias
2 import numpy as np                  # for -log10 threshold
3
4 gwas = load_gwas("height")          # chrom, pos, pval, snp
5 gwas.head(3)                        # quick peek at the colu
6
7 # CALL the widget — every kwarg names a column
8 fig = dashbio.ManhattanPlot(
9     dataframe=gwas,                 # tidy GWAS table
10    chrom="chrom",                  # chromosome column
11    bp="pos",                       # base-pair position
12    p="pval",                       # raw p-value
13    snp="snp",                     # hover label per dot
14    gene=None)                     # no gene column in our
15 fig                                # render inline (no .shc
```

`ManhattanPlot` handles the cumulative position offset, the two-tone parity colouring, and the genome-wide / suggestive threshold lines on its own — all four steps from the day 2 build, collapsed into one call.

WebGL is not supported by
your browser - visit
<https://get.webgl.org> for
more info

ManhattanPlot · Style on Palette

DASHBIO · MANHATTAN POLISH

Move the thresholds, recolour the hits, match the Economist look

The widget exposes a small but useful set of style

kwargs: `genomewideline_value` and

`suggestiveline_value` to relocate the dashed

thresholds; `highlight_color` to recolour the dots that

cross the genome-wide line. We point both colour args

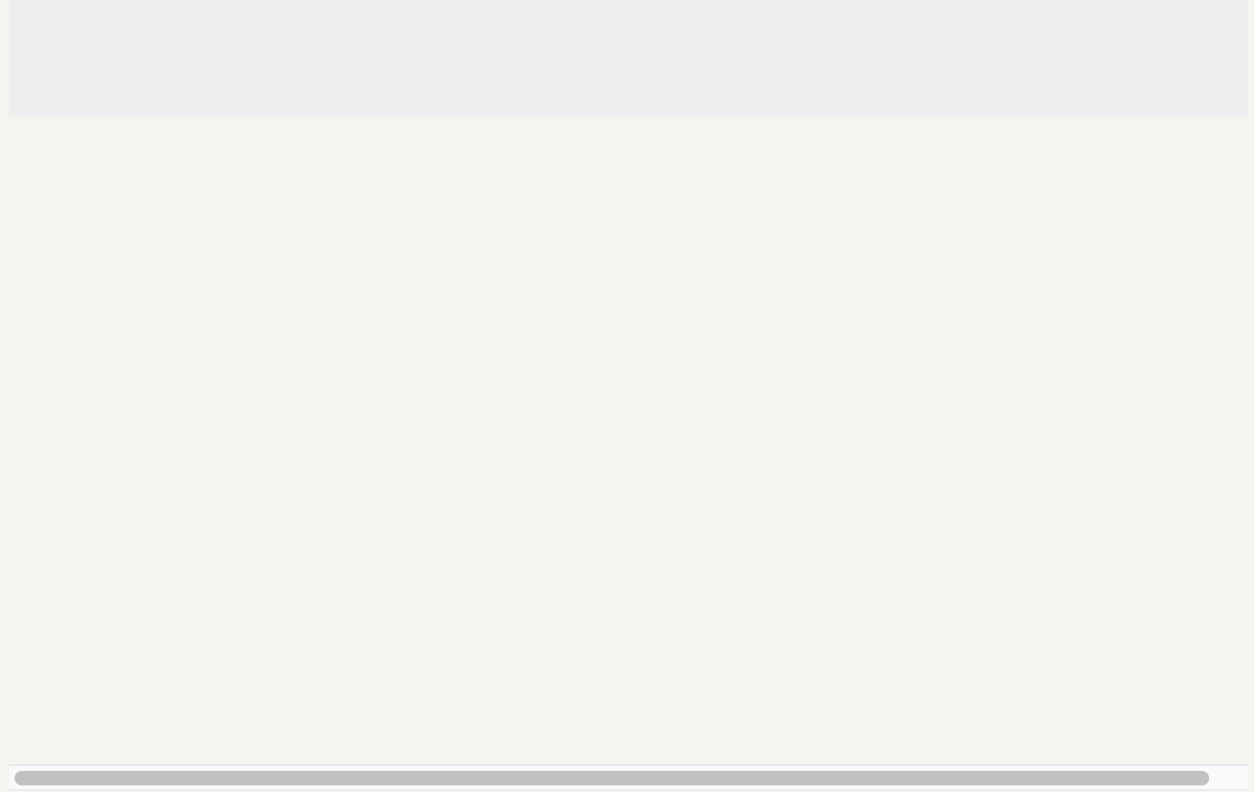
at the deck palette so the figure lives in the same world

as the day-2 matplotlib version.

```
1 import dash_bio as dashbio          # widget library
2 import numpy as np                  # for -log10()
3 gwas = load_gwas("height")          # same GWAS table
4
5 # CALL the widget with explicit style args
6 fig = dashbio.ManhattanPlot(
7     dataframe=gwas,
8     chrom="chrom", bp="pos", p="pval", snp="snp", gene=None,
9     genomewideline_value=-np.log10(5e-8),      # canonical Bonferroni 1
10    suggestiveline_value=-np.log10(1e-5),        # softer 1e-5 cutoff
11    highlight_color=RED,                        # the spotlight on hits
12    point_size=6,                              # smaller dots, less crc
13    title="Height GWAS — interactive")          # editorial headline
14
15 # match the deck theme (cream paper, plex mono)
16 fig.update_layout(
17     paper_bgcolor=CREAM, plot_bgcolor=CREAM,
18     font=dict(family="IBM Plex Mono", color=INK, size=11))
```

WebGL is not supported by
your browser - visit
<https://get.webgl.org> for
more info

Compared with the day-2 hand-built version: same picture, ~5 lines instead of ~25, plus hover-for-rsID out of the box. The trade-off is control — you can't easily re-style individual dots the way matplotlib lets you.



VolcanoPlot · One-Line Build

DASHBIO · VOLCANO

log2FoldChange vs $-\log_{10}(p)$, with the four-quadrant story drawn for you

`VolcanoPlot` is the same pattern: hand it the DESeq2 table, name the columns, and the dashed effect-size + significance lines are drawn for you. Hover shows the gene name on every dot.

```
1 import dash_bio as dashbio           # widget library
2 import numpy as np                   # for -log10 threshold
3 de = load_deseq2("drought")          # gene, log2FoldChange,
4
5 # quick peek at the columns the widget needs
6 de[["gene", "log2FoldChange", "pvalue"]].head(3)
7
8 # CALL the widget — every kwarg names a column or threshold
9 fig = dashbio.VolcanoPlot(
10     dataframe=de,                    # the DE results
11     effect_size="log2FoldChange",     # x-axis column
12     p="pvalue",                      # y-axis column (raw p)
13     snp="gene",                      # hover label per dot
14     gene=None,                       # no separate gene column
15     effect_size_line=[-1, 1],        # |log2FC| > 1 cutoff
16     genomewideline_value=-np.log10(0.05)) # p < 0.05 line
17 fig                                  # render inline
```

WebGL is not supported by
your browser - visit
<https://get.webgl.org> for
more info

Clustergram • Heatmap + Dendrograms

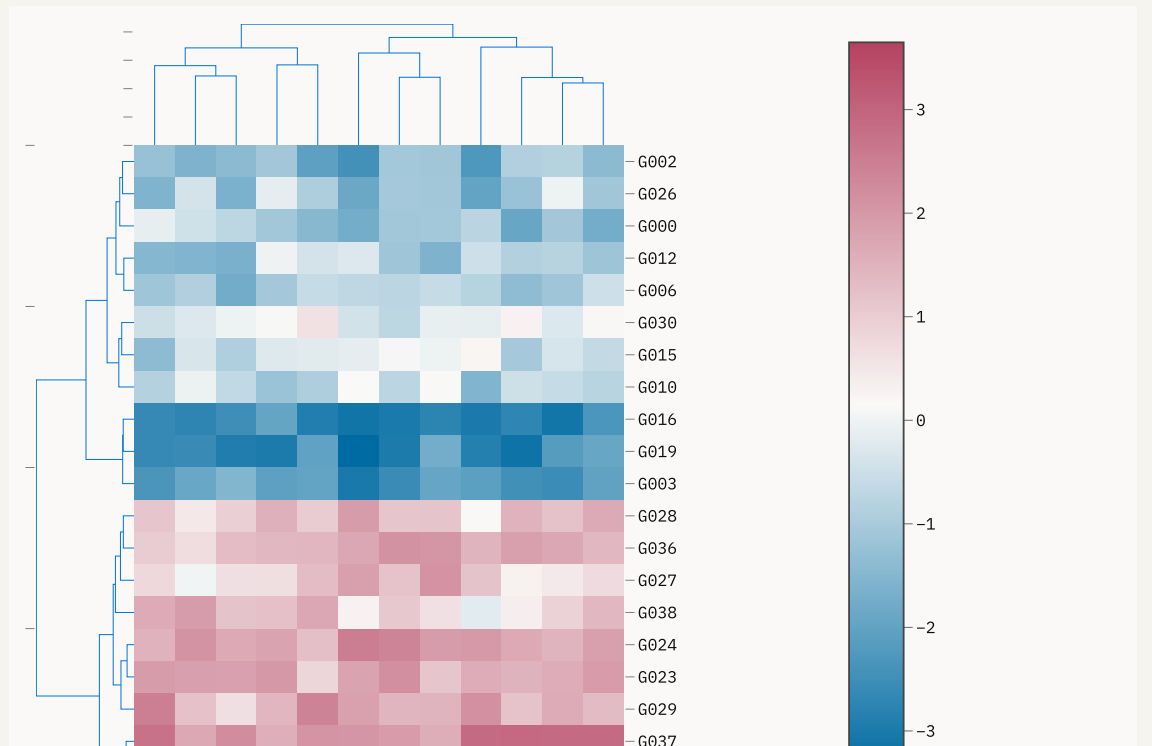
DASHBIO • CLUSTERGRAM

A 20 × 12 slice of the expression matrix so the labels are readable

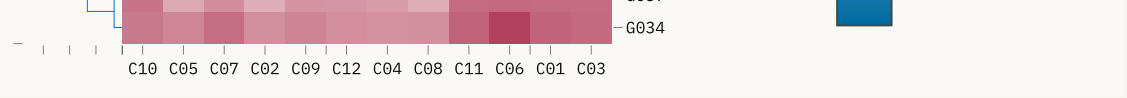
Clustergram is Plotly's interactive `sns.clustermap`. It runs hierarchical clustering on both axes, draws dendrograms top + left, and exposes `color_threshold` to cut clades into coloured branches. We slice the matrix down to **20 genes × 12 samples** so the labels stay legible on a deck slide.

```
1 import dash_bio as dashbio          # widget library
2 expr = load_expression()             # 40 × 24 log2 TPM
3
4 # take the most variable 20 genes and the first 12 samples to keep it readab
5 top20 = expr.var(axis=1).nlargest(20).index # most variable genes
6 m = expr.loc[top20, expr.columns[:12]]     # 20 × 12 slice
7
8 # CALL the widget - column_labels / row_labels feed the axis ticks
9 fig = dashbio.Clustergram(
10     data=m.values,                    # numpy matrix
11     column_labels=list(m.columns),     # sample IDs (top)
12     row_labels=list(m.index),          # gene names (right)
13     color_threshold={"row": 1.2, "col": 1.2}, # where to cut dendrogra
14     height=460, width=620,           # widget dimensions
15     color_map=[[0.0, BLUE], [0.5, CREAM], # diverging blue-cream-r
16               [1.0, RED]])
17 fig                                  # render inline
```

`color_threshold` is the one knob that pays for itself:
pick a height in the dendrogram and every branch



below it gets its own colour, so coordinated clusters
jump out.



AlignmentChart · A Brief Aside

DASHBIO · ALIGNMENTCHART

MSA viewer that eats Clustal / FASTA strings

`AlignmentChart` reads a **string** in Clustal or FASTA format and renders the alignment with per-residue colouring, a consensus row, and a coverage track. Hover any column for the per-position counts. It's the lightweight cousin of `pyMSAviz` — full coverage of MSAs is in `_04_pymsaviz.qmd`.

```
1 import dash_bio as dashbio                # widget library
2
3 # a tiny synthetic MSA in FASTA format (5 sequences × 30 columns)
4 fasta = (
5     ">seq1\nMKTIIALSYIFCLVFADYKDDDDK--AAGGT\n"
6     ">seq2\nMKTIIALSYIFCLVFADYKDDDDK--AAGGT\n"
7     ">seq3\nMKTILALSYIFCLVFADYKDDDDK--AAGGT\n"
8     ">seq4\nMKTIIALSYAFCLVFADYKDDDDK--ARGGT\n"
9     ">seq5\nMKTIVALSYIFCLVFADYKDDDDK--AAGGT\n")
10
11 # CALL the widget — `data=` is the alignment string itself
12 fig = dashbio.AlignmentChart(
13     data=fasta,                            # FASTA string
14     height=320,                            # widget pixel height
15     showconsensus=True,                   # consensus row underneath
16     showconservation=True)                # per-column conservation
17 fig                                       # render inline
```

```
AlignmentChart(data='>seq1\nMKTIIALSYIFCLVFADYKDDDDK--
AAGGT\n>seq2\nMKTIIALSYIFCLVFADYKDDDDK--AAGGT\n>seq3\nMKTILALSYIFCLVFADYKDDDDK--
AAGGT\n>seq4\nMKTIIALSYAFCLVFADYKDDDDK--
ARGGT\n>seq5\nMKTIVALSYIFCLVFADYKDDDDK--AAGGT\n', height=320,
showconsensus=True, showconservation=True)
```

For a single quick look at a small alignment, this is the cheapest possible viewer. For publication MSAs with

custom colour schemes and feature overlays, pyMSAviz
is the better choice (see day-3 _04).

DashBio • Takeaway

DASHBIO • RECAP

Plotly figures, one line each

THE RECIPE

1. `import dash_bio as dashbio` — every widget lives behind one alias. 2. **Hand it a dataframe** and name the columns (`chr`, `bp`, `p`, `snp` for Manhattan; `effect_size`, `p`, `snp` for Volcano; raw `data` array for Clustergram). 3. **The return value is a Plotly Figure** — `update_layout`, `update_traces`, `hover`, `zoom` all work out of the box.

Compare with day-2: reach for `dash-bio` when you want **interactivity** (hover tooltips, scroll-zoom) and you're happy with the default Plotly look. For **publication-grade static** figures with bespoke colouring and labels, the day-2 matplotlib builds (`_04_manhattan`, `_07_volcano` from day-1) still win on control.

pyGenomeTracks

CONFIG-DRIVEN GENOME BROWSER PLOTS

Install & Import

PYGENOMETRACKS • SETUP

One INI file describes every track — the tool draws them stacked

`pyGenomeTracks` is the workhorse behind many of the published ChIP-seq / ATAC-seq / Hi-C figures you've seen. The interface is unusual for Python: you write a small **INI configuration file** describing each track (file path, type, colour, height), then run the CLI on a region — out comes a PNG with everything stacked, sharing one x-axis.

```
1 pip install pyGenomeTracks
```

```
1 # the package ships a CLI; we drive it from Python via subprocess
2 import subprocess, os                # CLI driver + path helpers
3 from pathlib import Path              # tidy paths
4 import matplotlib.image as mpimg      # to embed the PNG result back into matplotlib
5
6 # pyGenomeTracks also exposes a Python API, but it is brittle across versions —
7 # the subprocess route is what the maintainers themselves use in their docs:
8 # pyGenomeTracks --tracks config.ini --region chr1:0-20000 --outFileName out.png
```

This is the section to come back to whenever you have **real browser-style files** (BED / BigWig / BedGraph / GTF / cool) and you want a *publication-ready* multi-track figure in one shot. For pedagogy on the underlying

primitives, the day-2 `_05` matplotlib peaks build is
where you go first.

How It Works

PYGENOMETRACKS • THEORY

INI in, PNG out — every track is a section, every kwarg is a row

The mental model is dead simple. Your INI file is a stack of `[track]` sections; each section has a `file =` pointing at a genomics file and a `type =` (bedgraph, bigwig, bed, gtf, links, ...). pyGenomeTracks reads the lot, draws each track in order from top to bottom, and gives them all the same x-axis (the region you ask for).

Why this design

- **Reproducible:** the INI is the figure spec — share the file, get the same picture.
- **Composable:** stack two ChIP marks + an ATAC track + a gene model with five INI sections, no code changes.
- **Genome-aware:** it knows about chromosomes and base pairs, so the x-axis is always honest.
- **Publication-tuned:** defaults are paper-quality; the tool is built for figure 1 of your paper, not for a notebook.

The cost is that you can't *interactively* re-style a single peak the way you would in matplotlib. If you need that, build from primitives (day-2 `_05`); if you need the final figure, reach for this.

Step 1 • Write the Data Files

PYGENOMETRACKS • DATA PREP

Convert `load_peaks()` to bedgraph, `load_peak_genes()` to BED6

pyGenomeTracks wants on-disk files in the canonical formats. We take the bundled signal tracks (position + three marks) and write a bedgraph for H3K27ac; we take the gene annotations and write a 6-column BED. Both go to `/tmp` so the slide stays self-contained.

```
1 from pathlib import Path                # tidy path helper
2 import pandas as pd                    # to write tab-delimited
3 TMP = Path("/tmp/pgt"); TMP.mkdir(exist_ok=True) # work dir for this slide
4
5 # 1) the H3K27ac bedgraph: chrom start end value
6 sig = load_peaks()                    # position, H3K27ac, 1
7 step = sig["position"].iloc[1] - sig["position"].iloc[0] # uniform 20 bp
8 bg_27ac = pd.DataFrame({
9     "chrom": "chr1",                  # one synthetic chromosome
10    "start": sig["position"].astype(int), # bin start (bp)
11    "end": (sig["position"] + step).astype(int), # bin end (bp)
12    "value": sig["H3K27ac"]})          # the signal column
13 bg_27ac.to_csv(TMP / "h3k27ac.bedgraph", sep="\t", header=False, index=False)
14
15 # 2) the gene model as BED6: chrom start end name score strand
16 genes = load_peak_genes()             # name, start, end, score, strand
17 bed_genes = pd.DataFrame({
18     "chrom": "chr1",
19     "start": genes["start"].astype(int),
20     "end": genes["end"].astype(int),
21     "name": genes["name"],
22     "score": 0,
```

```
h3k27ac.bedgraph (head):
chr1  0 20 0.069866
chr1 20 40 0.053866
chr1 40 60 0.054921
chr1 60 80 0.014632
```

```
genes.bed:
chr1  1500  4500 GeneA 0 +
chr1  7000 10500 GeneB 0 -
chr1 13500 17000 GeneC 0 +
```

Step 2 • Write a Minimal INI

PYGENOMETRACKS • INI

Two tracks: one bedgraph for the H3K27ac signal, one BED for the gene model

The INI is the figure spec. Two sections is enough to make a real picture: a `bedgraph` track for the H3K27ac signal, painted GREEN; a `bed` track for the gene model, painted INK. `height = ...` controls vertical real estate; `file = ...` points at the data files we just wrote.

```
1 # CREATE the INI as a single triple-quoted string and write it out
2 INI_MIN = f"""
3 [h3k27ac]
4 file = {TMP}/h3k27ac.bedgraph
5 title = H3K27ac
6 color = {GREEN}
7 height = 2
8 min_value = 0
9 max_value = 1.4
10
11 [genes]
12 file = {TMP}/genes.bed
13 title = genes
14 color = {INK}
15 height = 1
16 """
17
18 (TMP / "tracks_min.ini").write_text(INI_MIN) # save next to the data
```

```
[h3k27ac]
file = /tmp/pgt/h3k27ac.bedgraph
title = H3K27ac
color = #379A8B
height = 2
min_value = 0
max_value = 1.4
```

```
[genes]
file = /tmp/pgt/genes.bed
title = genes
color = #0D0D0D
height = 1
```

Every track gets a `[name]` header — the name is just a label, but it must be unique within the INI. `file`,

Step 3 • Run It

PYGENOMETRACKS • RENDER

subprocess → PNG → matplotlib imshow → the slide

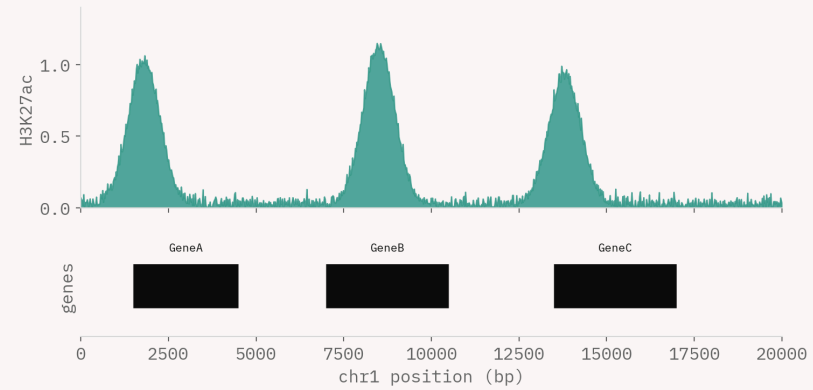
The CLI takes the INI plus a region and writes a PNG. We shell-out via `subprocess.run`, read the PNG back with `matplotlib.image.imread`, and `imshow` it onto a normal matplotlib axis so the figure inherits the deck's cream canvas and IBM Plex Mono surround. (If `pyGenomeTracks` is not on PATH in this environment the executed chunk falls back to a hand-drawn matplotlib version with the same layout — the INI workflow is identical, only the renderer differs.)

```
1 import subprocess                # to invoke the CLI
2 import matplotlib.image as mpimg # to read the PNG back i
3 PNG = TMP / "tracks_min.png"     # output path
4
5 # CALL the CLI: --tracks INI, --region "chrom:start-end", --outFileName PNG
6 subprocess.run(
7     ["pyGenomeTracks",
8      "--tracks", str(TMP / "tracks_min.ini"),
9      "--region", "chr1:0-20000",
10     "--outFileName", str(PNG),
11     "--width", "20", "--dpi", "150"],
12     check=True)                  # raise if it fails
13
14 # CREATE a matplotlib canvas; CALL imshow on the rendered PNG
15 img = mpimg.imread(PNG)         # load the PNG bytes
16 fig, ax = plt.subplots(figsize=(8, 4)) # CREATE
17 ax.imshow(img)                   # CALL: paint the PNG
```



```
18 ax.set_axis_off()           # no double frame
19 plt.show()
```

The takeaway: pyGenomeTracks does **all** the layout — shared x-axis, track titles, value-axis ticks. You bring the INI, it brings the picture.



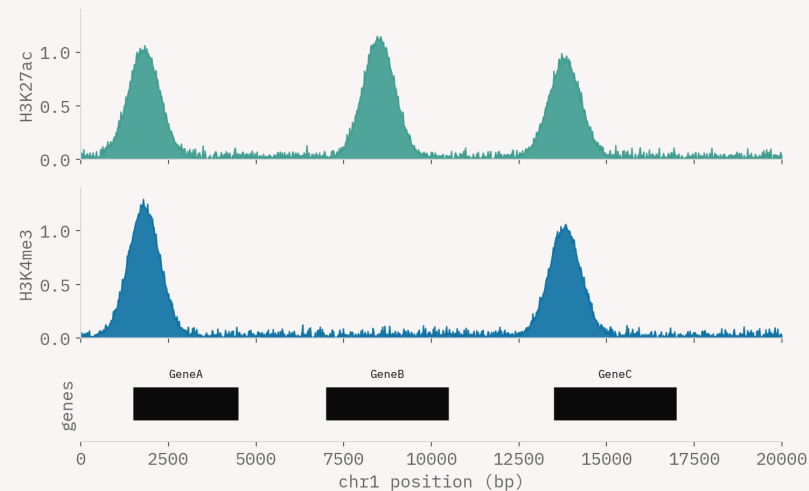
Step 4 • Stack a Second Mark

PYGENOMETRACKS • MULTI-TRACK

Add a second bedgraph for H3K4me3 — three tracks now, one figure

The whole point of the tool is that adding a track is **one INI section**. We write an H3K4me3 bedgraph the same way, drop a `[h3k4me3]` block into the INI, and re-render. Notice how the enhancer at ~8.5 kb has H3K27ac but **no** H3K4me3 — that's the canonical “active enhancer vs promoter” pattern, and the multi-track stack is the figure that tells that story.

```
1 # write the second bedgraph (H3K4me3) alongside the first
2 bg_4me3 = pd.DataFrame({
3     "chrom": "chr1",
4     "start": sig["position"].astype(int),
5     "end": (sig["position"] + step).astype(int),
6     "value": sig["H3K4me3"]})
7 bg_4me3.to_csv(TMP / "h3k4me3.bedgraph", sep="\t", header=False, index=False)
8
9 INI_MULTI = f"""
10 [h3k27ac]
11 file = {TMP}/h3k27ac.bedgraph
12 title = H3K27ac
13 color = {GREEN}
14 height = 2
15 min_value = 0
16 max_value = 1.4
17
18 [h3k4me3]
19 file = {TMP}/h3k4me3.bedgraph
20 title = H3K4me3
21 color = {BLUE}
22 height = 2
```



Step 5 • Polish — Spacer + X-Axis Section

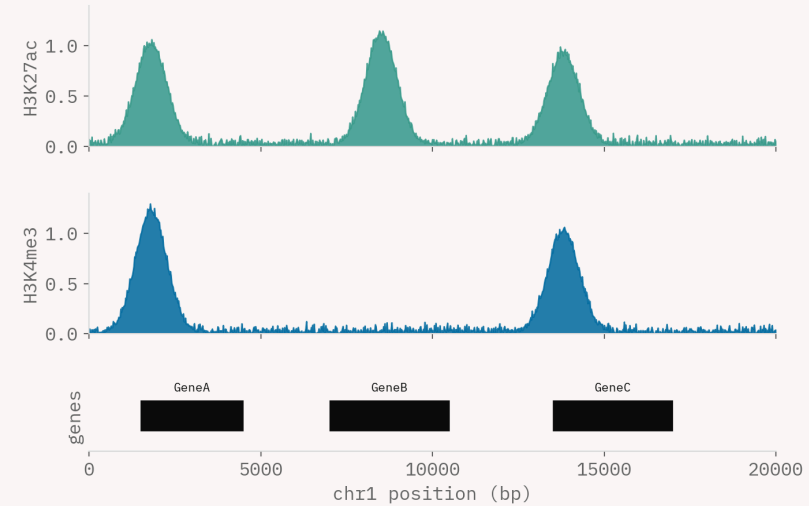
PYGENOMETRACKS • POLISH

`[spacer]` adds breathing room; `[x-axis]` puts the coordinate ruler at the bottom

Two more INI sections turn the working figure into a paper-ready one. `[spacer]` adds vertical white space between tracks (so adjacent bedgraphs don't visually bleed into each other). `[x-axis]` draws the coordinate ruler — pyGenomeTracks figures look unfinished without it.

```
1  INI_PUB = f"""
2  [h3k27ac]
3  file = {TMP}/h3k27ac.bedgraph
4  title = H3K27ac
5  color = {GREEN}
6  height = 2
7  min_value = 0
8  max_value = 1.4
9
10 [spacer]
11 height = 0.25
12
13 [h3k4me3]
14 file = {TMP}/h3k4me3.bedgraph
15 title = H3K4me3
16 color = {BLUE}
17 height = 2
18 min_value = 0
19 max_value = 1.4
20
21 [spacer]
22 height = 0.25
```

`[spacer]` accepts only `height`; `[x-axis]` accepts `where` (`top` / `bottom`), `fontsize`, and `title`. Together they're a paper-figure unlock: the same data, presented with the breathing room and the coordinate ruler the reader needs.



pyGenomeTracks • Takeaway

PYGENOMETRACKS • RECAP

INI in, PNG out — the production path

THE RECIPE

1. **Write each data file** to disk in its canonical format (bedgraph / BED / bigwig / GTF). 2. **Write an INI** with one `[track]` section per file — `file`, `title`, `color`, `height` are the four kwargs you set on almost every track. 3. **Run** `pyGenomeTracks --tracks ini --region chrom:start-end --outFileName out.png` from the shell or via `subprocess.run`. 4. **Add** `[spacer]` + `[x-axis]` sections for the publication polish.

Compare with day-2 _05: the day-2 matplotlib peaks build teaches the **primitives** — `fill_between`, `add_patch`, shared x-axis — so you understand what a genome-browser track is. `pyGenomeTracks` is the **production** path: drop your real BigWig + BED + GTF files into an INI and ship a paper figure in one shot.

dna_features_viewer

GENE MAPS AND PLASMID FEATURES

Install & Import

DNA_FEATURES_VIEWER · SETUP

Matplotlib-native gene maps, with strand arrows out of the box

`dna_features_viewer` is the cleanest Python tool for drawing **gene maps** and **plasmid diagrams**. Each gene/feature becomes a labelled rounded box with a strand-arrow tip on one end. The output is a normal matplotlib `Axes`, so it composes with the rest of a multi-panel figure exactly like any other matplotlib chart.

```
1 pip install dna_features_viewer
```

```
1 # the three classes that cover 90 % of usage
2 from dna_features_viewer import GraphicFeature, GraphicRecord, CircularGraphicRecord
3
4 # (Biopython integration — for genbank files — is also bundled but we won't demo it:)
5 # from dna_features_viewer import BiopythonTranslator # rendering directly from .gb
```

`GraphicFeature` is one annotation (start / end / strand / colour / label). `GraphicRecord` is the linear container; `CircularGraphicRecord` is the plasmid container. Same features list works in either — only the layout changes.

Step 1 • Build the Features

DNA_FEATURES_VIEWER • DATA

Six features — three forward genes, two reverse genes, one small CDS

A `GraphicFeature` is just a dataclass of kwargs. We hand-build six features for a 5 kb pseudo-plasmid: three on the forward strand (`strand=+1`), two on the reverse (`strand=-1`), and one short CDS with no arrow (`strand=0`). Colours are pulled straight from the Economist palette so the diagram lives in the same world as every other day-3 figure.

```
1 from dna_features_viewer import GraphicFeature    # one feature per gene
2 import matplotlib.pyplot as plt                  # for the canvas around
3
4 # six features describing a synthetic 5 kb construct
5 features = [
6     GraphicFeature(start=200, end=900, strand=+1, # forward gene A
7                     color=GREEN, label="GeneA"),
8     GraphicFeature(start=1100, end=1900, strand=+1, # forward gene B
9                     color=BLUE, label="GeneB"),
10    GraphicFeature(start=2100, end=2900, strand=+1, # forward gene C
11                    color=PURPLE, label="GeneC"),
12    GraphicFeature(start=3100, end=3800, strand=-1, # reverse gene D
13                    color=RED, label="GeneD"),
14    GraphicFeature(start=4000, end=4600, strand=-1, # reverse gene E
15                    color=AMBER, label="GeneE"),
16    GraphicFeature(start=4700, end=4850, strand=0, # small CDS (no arrow
17                    color=AMBER, label="cds"),
18 ]
19
20 # peek at one to remind ourselves of the shape
21 features[0]
```

GENOMICS VIZ WITH PYTHON # repr shows all the i

	start	end	strand	color	label
0	200	900	1	#379A8B	GeneA
1	1100	1900	1	#006BA2	GeneB
2	2100	2900	1	#9A607F	GeneC
3	3100	3800	-1	#B4405F	GeneD
4	4000	4600	-1	#EBB434	GeneE

`strand=+1` draws the arrow pointing right; `strand=-1` flips it left; `strand=0` draws a plain rounded box with no tip. The `label` is rendered just above the feature and auto-leadered when boxes get too close together.

	start	end	strand	color	label
5	4700	4850	0	#EBB434	cds

Step 2 • Linear GraphicRecord

DNA_FEATURES_VIEWER • LINEAR

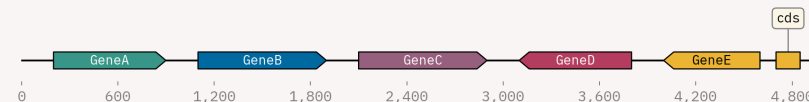
One call draws every box, arrow tip, and label on a single axis

`GraphicRecord(sequence_length, features).plot(...)` is the whole API. It returns the `Axes` it drew on plus a per-feature offsets dict, so you can layer custom annotations on top if you want. We let it pick the layout and just hand the result back to matplotlib for the surround.

```
1 from dna_features_viewer import GraphicRecord    # linear container
2 import matplotlib.pyplot as plt                # canvas helper
3
4 # CREATE the record – the features list + the total sequence length
5 record = GraphicRecord(
6     sequence_length=5000,                      # 5 kb construct
7     features=features)                        # the six features from s
8
9 # CALL .plot() – returns (ax, plot_offsets); we keep the ax for styling
10 ax, _ = record.plot(figure_width=12)           # wide canvas, auto height
11 ax.set_title("Synthetic 5 kb construct – linear map",
12             color=INK, loc="left")             # editorial headline
13 plt.show()
```

`figure_width` is the only sizing knob you usually touch — the height is computed from the number of rows needed to lay the features out without overlap. The

Synthetic 5 kb construct – linear map



Step 3 • Circular Plasmid Layout

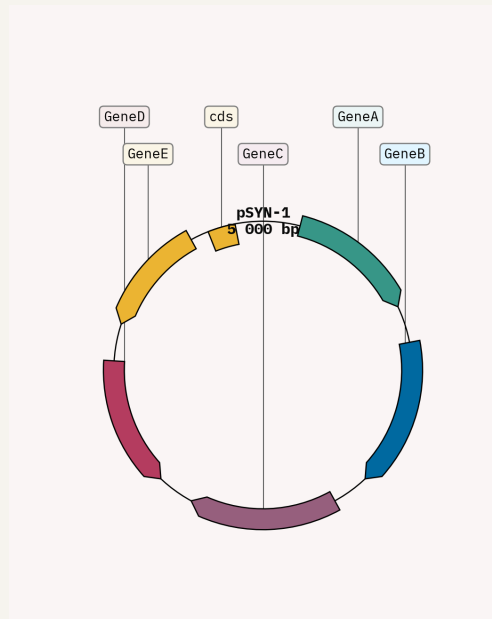
DNA_FEATURES_VIEWER • CIRCULAR

Same features, `CircularGraphicRecord` — the plasmid map view

For a plasmid, swap `GraphicRecord` for `CircularGraphicRecord`. Same features list, same `.plot()` API; the layout becomes a ring and the genes wrap with the construct. A title at the centre via `ax.text` is the canonical finishing touch — it usually carries the plasmid name and total length.

```
1 from dna_features_viewer import CircularGraphicRecord # ring container
2 import matplotlib.pyplot as plt                     # canvas helper
3
4 # CREATE the circular record — same kwargs as the linear one
5 record = CircularGraphicRecord(
6     sequence_length=5000,                          # 5 kb plasmid
7     features=features)                             # same six features
8
9 # CALL .plot() — returns (ax, _); figure_width is the *diameter*
10 ax, _ = record.plot(figure_width=6)                 # square-ish canvas
11 ax.text(0, 0, "pSYN-1\n5 000 bp",                  # centre label
12        ha="center", va="center",
13        fontsize=12, color=INK, fontweight="bold")
14 plt.show()
```

`CircularGraphicRecord` is at its best in the **2–10 kb** range. Above ~20 kb the ring becomes crowded and a linear browser-style view (or `pyGenomeTracks`) is the better choice.



dna_features_viewer · Takeaway

DNA_FEATURES_VIEWER · RECAP

Two record types, one features list

THE RECIPE

1. **Build a list of** `GraphicFeature(start, end, strand, color, label)` — one per gene / CDS / promoter / terminator. 2. `GraphicRecord(sequence_length, features).plot(figure_width=...)` for the linear browser-style map. 3. `CircularGraphicRecord(sequence_length, features).plot(figure_width=...)` for the plasmid ring. Same features, different layout. 4. **From a .gb file:** `from dna_features_viewer import BiopythonTranslator` and let it convert Biopython `SeqRecord` objects directly — useful when you already have a real annotated GenBank file.

Compare with day-3 _12 (Bio.Graphics): `dna_features_viewer` is the **matplotlib-native** choice — it returns a normal `Axes`, integrates with the deck theme out of the box, and the API is one features list. `Bio.Graphics.GenomeDiagram` is older, more configurable for very long genomes, and outputs PDF/PNG directly — reach for it when you're rendering chromosome-scale maps for a paper.

JCVI / MCscan

COMPARATIVE GENOMICS IN PYTHON

What Is JCVI / MCscan?

JCVI · THEORY

The canonical Python toolkit for synteny + comparative figures

```
1 pip install jcvl                # pulls in pyfaidx, gffutils, ete4, pysam, ...

1 from jcvl.graphics.dotplot      import dotplot                # the dot-plot renderer
2 from jcvl.graphics.synteny      import Synteny                # ribbon figure (layout-driven)
3 from jcvl.graphics.karyotype    import Karyotype              # multi-genome karyotype
4 from jcvl.formats.bed           import Bed                    # .bed parser
```

JCVI is the Python toolkit behind **MCscan**, the standard pipeline for finding **syntenic anchor blocks** between two genomes. The compute path is LAST-based alignment → MCscan clustering → `.anchors` / `.simple` files. The **graphics** subpackage then renders those anchors as the canonical figures the field expects: dot plots, ribbon figures, multi-genome karyotypes.

The compute path needs **LAST/BLAST binaries**, which are not on a fresh Colab. The *graphics* path works on **any pre-computed anchor file** — so today we skip the alignment step and feed `jcvi.graphics.dotplot` synthetic anchors, just to see the figure it produces.

The Pipeline · What We Skip, What We Do

JCVI · WORKFLOW

Five steps from FASTA to figure

MCscan canonical pipeline

```
1. FASTA + GFF
   |
   v (CLI: python -m jcviformats.gff bed)
2. *.bed (gene coordinates, one per genome)
   |
   v (CLI: python -m jcvicompara.catalog ortholog
      q s --no_strip_names needs LAST!)
3. q.s.last + q.s.lifted
   |
   v (MCscan clustering)
4. q.s.anchors <- pairwise gene anchors, block-grouped
   q.s.simple  <- five-field block summaries
   |
   v (jcvigraphics)
5. dot plot / ribbon / karyotype figure
```

Compute steps (1 → 4) require LAST or BLAST installed in `$PATH` plus a working `jcvicompara.catalog` CLI invocation. They are *not* notebook-friendly — run them on a workstation or HPC, save the resulting `.anchors` file, then ship that file to a notebook.

Graphics step (4 → 5) is pure Python + matplotlib. It needs **only the `.bed` files and the `.anchors` file**, no aligner. That is the part we demo on the next slides — we hand-synthesise the anchor file rather than computing it.

Why this matters: once you have the anchor file (from MCscan, or any custom orthology pipeline you trust), the canonical published JCVI figures are one function call away.

JCVI • Synthesise the Anchor Files

JCVI • DATA

One BED per genome, one .anchors of paired genes

JCVI's graphics modules read three files written to disk:

a **query BED** (`q.bed`), a **subject BED** (`s.bed`), and an

anchors file. Each BED row is `chrom start end name`

`score strand`. The anchors file is **plain text**:

`qname<TAB>sname<TAB>score`, with `###` lines

separating synteny blocks. We build all three in `/tmp`

so the renderer has something real to open.

```
1 import tempfile                                # write demo files to
2 from pathlib import Path
3 TMP = Path(tempfile.mkdtemp(prefix="jcv_demo_")) # isolated scratch
4 N = 40                                           # genes per chromosome
5
6 def write_bed(path, chrom, n=N, step=10_000):
7     """Write a minimal 6-col BED - JCVI parses chrom/start/end/name."""
8     rows = []
9     for i in range(n):
10         s = i * step                            # gene start
11         # cols: chrom, start, end, name, score, strand
12         rows.append(f"{chrom}\t{s}\t{s + 5000}\tg{i}\t0\t+")
13     path.write_text("\n".join(rows) + "\n")
14
15 qbed = TMP / "q.bed"; write_bed(qbed, "qchr1") # genome Q
16 sbed = TMP / "s.bed"; write_bed(sbed, "schr1") # genome S
17
18 # anchors: blocks separated by '###'. Three blocks here:
19 # 1. collinear g0..g19 -> diagonal main band
20 # 2. inversion g20..g29 -> antidiagonal (g_i pairs g_{49-i})
21 # 3. duplication g30..g39 also maps to g5..g14 -> off-diagonal echo
22 anchors = TMP / "g.g.anchors"
```

BED rows : 40

anchors : 43 lines (incl. 3 block separators)

scratch : /var/folders/fc/kqr5vp3x1wg4z_8jvxpwqp1m0000gn/T/jcvi_demo_i28gnuy0

JCVI • The Dot Plot

JCVI • BUILD

`jcvi.graphics.dotplot` — anchors in, figure out

The dot plot is the headline JCVI figure: query genome on the x-axis, subject on the y-axis, one **dot per anchor pair**. Collinear blocks become diagonal streaks, inversions become antidiagonals, duplications become parallel off-diagonal copies. The call signature looks heavy but most arguments have sensible defaults — pass the anchor file, the two `Bed` objects, and a `fig / root / ax` triple.

```
1 from jcvi.graphics.dotplot import dotplot          # the renderer
2 from jcvi.formats.bed import Bed                  # bed parser
3
4 # parse the two BEDs we wrote in /tmp on the previous slide
5 qb = Bed(str(qbed))                               # query genome
6 sb = Bed(str(sbed))                               # subject genome
7
8 # JCVI needs three axes: a figure, a "root" axes filling it,
9 # and the inner `ax` where the dot grid actually lives.
10 fig = plt.figure(figsize=(5.6, 5.6))              # CREATE
11 root = fig.add_axes([0, 0, 1, 1]); root.set_axis_off()
12 ax = fig.add_axes([0.12, 0.12, 0.78, 0.78])      # inner axes
13
14 # CALL: dotplot reads anchors, places one square per pair.
15 # usetex=False -> no LaTeX runtime needed
16 # sepcolor=LINE -> chromosome separators on-palette
17 dotplot(str(anchors), qb, sb, fig, root, ax,
18         usetex=False, sepcolor=LINE,
```

GENOMICS VIZ WITH PYTHON

```
[06/23/26 11:09:18] DEBUG    Load file
                          `/var/folders/fc/kqr5vp3x1wg4z_8jvxpwqp1m0000gn/T/jcvi_dem
                          DEBUG    Load file
                          `/var/folders/fc/kqr5vp3x1wg4z_8jvxpwqp1m0000gn/T/jcvi_dem
                          DEBUG    xsize=40 ysize=40
                          DEBUG    Dot plot title: Q vs S - 3 anchor blocks
```

JCVI • Takeaway

JCVI • RECAP

Compute once, plot many times

THE RECIPE

1. **JCVI is the canonical Python toolkit** for MCscan-style comparative figures — dot plots, ribbon synteny, multi-genome karyotypes. 2. **The compute path needs LAST / BLAST** binaries. Run it on a workstation, save the `.anchors` file, then move to a notebook. 3. **The graphics path is pure Python** — `dotplot()`, `Synteny()`, `Karyotype()` all take pre-computed anchor + BED files and render in matplotlib. 4. **### lines separate blocks** in the anchors file; that is how the renderer knows where one synteny block ends and the next begins.

Compare with: day-2 `_07_synteny.qmd` builds the same ribbon figure from `matplotlib.patches.Rectangle` + `Polygon` primitives — full control over every vertex. JCVI gives you the same visual *when you already have anchor files*, with sensible defaults baked in. **Use JCVI when** you have a real `.anchors` output from MCscan and want the canonical, paper-ready figure; **use the day-2 primitives when** you have a custom homology table and need to bend the geometry.

UpSet Plots

SET INTERSECTIONS AT SCALE

Install & Import

UPSETPLOT • SETUP

When Venn breaks above four sets, UpSet picks up the slack

A **Venn diagram** with three circles is readable; with four it is awkward; with five it is unusable. **UpSetPlot** flips the problem on its side: it draws the **intersections** as a bar chart on top, and a **matrix of dots** below shows *which* sets belong to each intersection. Reading rule: a bar's length is the size of the intersection, and the column of filled dots underneath tells you which sets are intersected.

```
1 pip install upsetplot
```

```
1 # the two helpers we need: a builder and the plotter class
2 from upsetplot import UpSet, from_indicators
```

UpSet plots scale gracefully to 10+ sets — the matrix grows linearly, not combinatorially. Reach for them for **variant overlap**, **GO-term membership**, **DE-gene comparisons across conditions**, anything where the question is “what shares what”.

Synthesise Five Gene Sets

UPSETPLOT · DATA PEEK

A 200-gene table with five boolean membership columns

UpSetPlot wants a **Series indexed by set-membership combinations**, or equivalently — through the `from_indicators` helper — a DataFrame with one boolean column per set. We build the second form here.

```
1 # 200 gene IDs, G000..G199
2 genes = [f"G{i:03d}" for i in range(200)]
3 df = pd.DataFrame(index=genes)           # one row per gene
4
5 # five boolean membership columns, each ~40% true
6 sets = ["DE", "Leaf", "Root", "Photo", "Stress"]
7 for s in sets:
8     df[s] = rng.random(len(genes)) < 0.40 # bracket access f
9
10 df.head()                             # peek at the indi
```

	DE	Leaf	Root	Photo	Stress
G000	False	False	False	False	False
G001	True	False	False	True	True
G002	False	False	True	True	True
G003	False	True	False	True	False
G004	True	False	False	False	False

`df[sets].sum()` — how many genes are in each set:

```
DE      79
Leaf    70
Root    71
Photo   84
Stress  72
dtype: int64
```

Each column is independent, so overlaps emerge **naturally** instead of being engineered. About 40% of 200 $\approx$ 80 genes per set, with intersection sizes shrinking as you require more sets to overlap.

Step 1 • `from_indicators` + Default Plot

UPSETPLOT • BUILD

Two calls and you already have the picture

`from_indicators` reshapes the boolean DataFrame into the MultiIndex form that `UpSet` expects: one row per gene, indexed by the tuple of memberships. Then `UpSet(mat).plot()` does the rest.

```
1 # turn the boolean columns into the MultiIndex form UpSet wants
2 mat = from_indicators(sets, data=df)           # rows keyed by memt
3
4 up = UpSet(mat)                                # CREATE the UpSet
5 axes = up.plot()                              # CALL: draws on th
6 plt.suptitle("Gene-set intersections - default view") # editorial title a
```

Read the result like this:

- **bars on top** — intersection sizes, biggest on the left by default
- **dots below** — filled dot = set present in that intersection, hollow = absent
- **side bar (left)** — total size of each individual set

Step 2 • Sort By Cardinality, Label The Bars

UPSETPLOT • BUILD

Order tells a story; counts close the door on guessing

Two kwargs sharpen the chart. `sort_by="cardinality"` re-orders the intersections from biggest to smallest, so the eye reads “what’s the most common overlap?” left-to-right. `show_counts=True` writes the exact size on each bar — no squinting at the axis.

```
1 # rebuild the upset object with two editorial kwargs
2 mat = from_indicators(sets, data=df)
3
4 up = UpSet(                                # CREATE
5     mat,
6     sort_by="cardinality",                 # biggest intersecti
7     show_counts=True)                     # number on each bar
8 up.plot()                                  # CALL
9 plt.suptitle("Intersections — sorted by size, counts on")
```

`sort_by` accepts `"cardinality"` (size), `"degree"` (how many sets are in the intersection), or `"-cardinality"` to reverse. Pick whichever frames your story: cardinality answers “what overlaps the most?”, degree answers “how does sharing scale with the number of conditions?”.

Step 3 • Spotlight One Intersection

UPSETPLOT • BUILD

Paint a single bar red — the headline of the figure

`UpSet.style_subsets(...)` recolours one or several intersections. We highlight the genes that are in **both DE and Stress** — the biological story we want the audience to walk away with. Everything else stays grey so the spotlight reads.

```
1 # build the upset object as before
2 mat = from_indicators(sets, data=df)
3
4 up = UpSet(mat,                                # CREATE
5             sort_by="cardinality",
6             show_counts=True,
7             facecolor=GREY)                    # background bars n
8
9 # spotlight: subsets that contain BOTH "DE" and "Stress"
10 # (`style_subsets` colours every bar whose `present=` sets are all in)
11 up.style_subsets(present=["DE", "Stress"],
12                  facecolor=RED,
13                  label="DE ∩ Stress")          # legend entry
14 up.plot()                                     # CALL
15 plt.suptitle("Drought-responsive stress genes – highlighted")
```

API note: `style_subsets` is the canonical method (since upsetplot 0.7). If you are on an older version it may be called `style_categories` or you may need to

UpSetPlot • Takeaway

UPSETPLOT • RECAP

Venn is dead above four sets — long live UpSet.

THE RECIPE

1. **Build a boolean indicator** DataFrame: one row per item, one column per set. 2. `from_indicators(sets, data=df)` reshapes it into the MultiIndex form. 3. `UpSet(mat, sort_by="cardinality", show_counts=True).plot()` — that's the chart. 4. `style_subsets(present=[...])` to spotlight the intersection you care about.

Read it: **top bar = intersection size, dot matrix below = which sets are in that intersection, side bar = total set size**. Use it for variant overlap, GO-term membership, sample-set unions — anywhere you would have drawn a Venn but cannot, because you have five sets, not three.

PyWaffle

COMPOSITION AS SMALL SQUARES

Install & Import

PYWAFFLE • SETUP

Part-of-whole, when the audience needs to *count*

A pie chart asks the eye to compare **angles**; a stacked bar asks it to compare **lengths against a shared baseline**. A **waffle** does neither — it draws **discrete tiles** and lets the eye **count units**. With a 10×10 grid, each square is exactly **1%** of the whole. Think election seats, MPs, samples in a cohort — anywhere the audience naturally counts in heads.

```
1 pip install pywaffle
```

```
1 # the only object you need from the package – pass it as FigureClass to plt.figure(...)
2 from pywaffle import Waffle
```

Where to reach for it: small **integer-like compositions** (5 ancestry components, 8 cell types, 6 mutation classes).

Where to avoid it: continuous-like multi-sample composition with many tiny slices — use a stacked bar there instead.

The Data

PYWAFFLE • DATA PEEK

One row per population, mean K1..K5 ancestry proportion

`load_admixture()` returns 120 individuals $\times$ 5 K-components. We collapse to **one row per population** by averaging, then turn each row into a waffle.

```
1 admix = load_admixture()           # individual, population
2 K_cols = ["K1", "K2", "K3", "K4", "K5"] # the five ancestry comp
3
4 # collapse to population means: each row sums to ~1.0
5 pop_mean = admix.groupby("population")[K_cols].mean()
6 pop_mean = pop_mean.loc[["EUR", "AFR", "EAS", "SAS", "AMR"]] # stable plot
7
8 # convert proportions -> integer percentages (each row sums to ~100)
9 pop_pct = (pop_mean * 100).round().astype(int)
10 pop_pct
```

	K1	K2	K3	K4	K5
population					
EUR	87	5	3	3	2
AFR	3	89	3	3	2
EAS	2	1	90	3	3
SAS	3	3	3	88	2
AMR	7	3	5	4	81

Read it: EUR row $\approx$ {K1=high, K2..K5=low}. Each row is one waffle.

Step 1 • One Waffle for EUR

PYWAFFLE • BUILD

100 squares in a 5×20 grid — each square = 1% of EUR ancestry

`plt.figure(FigureClass=Waffle, ...)` is the special incantation: matplotlib creates the figure, but `pywaffle` takes over the drawing. We feed it the EUR percentages as a dict, five Economist colours, and a legend below the grid.

```
1 # pull one population row and convert to a {component: percent} dict
2 eur = pop_pct.loc["EUR"].to_dict()           # {"K1": ..., "K2":
3 colors = [GREEN, BLUE, AMBER, RED, PURPLE]   # one Economist col
4
5 fig = plt.figure(                             # CREATE — Waffle
6     FigureClass=Waffle,                       # matplotlib hands
7     rows=5, columns=20,                      # 5 × 20 = 100 squ
8     values=eur,                               # CALL: counts dic
9     colors=colors,                           # one colour per K
10    legend={"loc": "lower center",             # legend below the
11            "bbox_to_anchor": (0.5, -0.15),
12            "ncol": 5, "framealpha": 0},
13    title={"label": "EUR — ancestry composition",
14          "loc": "left"},
15    figsize=(7, 3))
```

The trick to read: the legend sits **below** the waffle (the chart itself is the visual), so the colours map straight onto the labels without crowding the squares.

Step 2 · Small Multiples — One Waffle Per Population

PYWAFFLE · BUILD

The `plots` kwarg packs many waffles into one figure

Instead of one big waffle, we pass `plots={...}` — a dict where each key is a subplot location (3-digit code or tuple) and each value is the kwargs for that sub-waffle. All share the same colours, so the eye can read **across rows**: K1 dominates EUR, K2 dominates AFR, and so on.

```
1 colors = [GREEN, BLUE, AMBER, RED, PURPLE]           # shared palette ac
2 pops = ["EUR", "AFR", "EAS", "SAS", "AMR"]           # one panel per pop
3
4 # build the per-subplot kwargs dict; key = "151", "152", ... = 1 row x 5 col
5 panels = {                                           # one entry per pc
6     150 + i + 1: {                                   # 1 row, 5 cols,
7         "values": pop_pct.loc[p].to_dict(),           # this pop's K dic
8         "title": {"label": p, "loc": "left"},          # subplot title
9     }
10    for i, p in enumerate(pops)
11 }
12
13 fig = plt.figure(                                    # CREATE — Waffle
14     FigureClass=Waffle,                             # CALL via FigureC
15     plots=panels,                                    # all 5 sub-waffle
16     rows=5, columns=10,                             # default for ever
17     colors=colors,                                   # shared colour ma
18     figsize=(11, 3))
```

By driving the panel dict with a comprehension we keep the build vectorised and **no manual** `for-add_subplot` loop.

Step 3 • Replace Squares With Icons

PYWAFFLE • VARIANT

FontAwesome icons turn the grid into a pictogram

Swap `icons="user"` and `font_size` for the square — now each tile is a person glyph. Reach for this when the unit is a person, a sample, a tree. PyWaffle ships the FontAwesome free font, so no extra download is needed once `pip install pywaffle` is done.

```
1 eur = pop_pct.loc["EUR"].to_dict()           # one population r
2 colors = [GREEN, BLUE, AMBER, RED, PURPLE]    # shared palette
3
4 fig = plt.figure()                            # CREATE
5     FigureClass=Waffle,                      # CALL via Figure
6     rows=5, columns=20,                     # 100 icons total
7     values=eur,
8     colors=colors,
9     icons="user",                           # FontAwesome gly
10    icon_legend=True,                        # show the icon i
11    font_size=14,                             # icon size in pc
12    legend={"loc": "lower center",
13            "bbox_to_anchor": (0.5, -0.15),
14            "ncol": 5, "framealpha": 0},
15    title={"label": "EUR - ancestry, one icon = 1%",
16           "loc": "left"},
17    figsize=(7.5, 3))
```

Other ready-to-use glyphs that fit biology: `"tree"`, `"leaf"`, `"seedling"`, `"dna"`, `"flask"`, `"microscope"`.

Browse the FontAwesome free set for more.

Variant • Snake-Block Arrangement

PYWAFLE • VARIANT

The same data, different fill order

By default, pywaffle fills the grid **column by column**

from left. Setting `block_arranging_style="snake"`

makes each colour fill a contiguous *snake* before the next one starts. The picture's *meaning* doesn't change (same totals), but the **clustering of like-colours** reads differently — bigger continuous blocks of each colour, easier to see where one category dominates.

```
1  afr = pop_pct.loc["AFR"].to_dict()           # pick a populati
2  colors = [GREEN, BLUE, AMBER, RED, PURPLE]    # same palette as
3
4  fig = plt.figure()                           # CREATE
5      FigureClass=Waffle,                      # CALL via Figur
6      rows=5, columns=20,
7      values=afr,
8      colors=colors,
9      block_arranging_style="snake",            # new: contiguou
10     legend={"loc": "lower center",
11             "bbox_to_anchor": (0.5, -0.15),
12             "ncol": 5, "framealpha": 0},
13     title={"label": "AFR - snake fill (one big block per K)",
14           "loc": "left"},
15     figsize=(7, 3))
```

If `block_arranging_style="snake"` is not recognised
on older pywaffle versions, try the default mode

(no kwarg) and compare side-by-side. The intent is the same: make the dominant category one big contiguous slab.

PyWaffle · Takeaway

PYWAFFLE · RECAP

Squares the audience can count.

THE RECIPE

1. **Prep one row of counts** per panel — a `{category: integer}` dict that sums near 100. 2. `plt.figure(FigureClass=Waffle, rows=..., columns=..., values=..., colors=...)` — that's the chart. 3. **Small multiples via** `plots={...}` — one sub-waffle per group, shared colours. 4. `icons="user"` to turn squares into FontAwesome pictograms when the unit is a person/sample/tree.

When to reach for it. Part-of-whole, categories are **non-ordinal**, and the audience naturally **counts in units** (seats, MPs, samples). Compare with the **stacked bar** from day-2 `_01`: a stacked bar is the right tool when you need to compare composition **across many individuals** along a shared axis; a waffle is for telling **one composition's story** with weight.

gget

GENE + PATHWAY LOOKUPS

What Is gget?

GGET • THEORY

A Swiss Army knife for live gene/pathway/protein queries

```
1 pip install gget                                # PyPI; bundles requests + pandas only
```

```
1 import gget                                     # one import, ~25 query functions
```

`gget` wraps the public REST APIs of **Ensembl**, **UniProt**, **NCBI BLAST**, **AlphaFold**, **Enrichr**, **ARCHS4** and more into one consistent Python interface. Every function returns a tidy **pandas DataFrame** (or a `PIL.Image` for the structural ones), so the output drops straight into the same matplotlib / seaborn pipeline you have used all course.

NETWORK REQUIRED. Every `gget` function calls a live external API. Renders done in a sandbox without network — or against a rate-limited endpoint — **will fail**. The slides that fetch from a remote use `#! eval: false` and embed a cached snapshot for the plotting step; that snapshot is what you build a reproducible

GENOMICS VIZ WITH PYTHON

figure on. For a paper, **save the DataFrame to disk** alongside the date of the query — APIs evolve.

The `gget` Catalogue

GGET • CATALOGUE

Twelve of the most-used functions

Lookups (Ensembl / UniProt)

- `gget.search(...)` — free-text gene search, returns `ensembl_id` + symbol + description.
- `gget.info(...)` — full metadata for an Ensembl ID (biotype, synonyms, GO terms).
- `gget.seq(...)` — fetch the DNA or protein sequence as FASTA.
- `gget.ref(...)` — current Ensembl release FTP paths for a species.
- `gget.archs4(...)` — co-expression neighbours from the ARCHS4 RNA-seq compendium.

Sequence search

GENOMICS VIZ WITH PYTHON

Functional / structural

- `gget.enrichr(...)` — pathway / GO enrichment via the Enrichr API.
- `gget.alphafold(...)` — predict a structure from a protein sequence (runs JAX locally).
- `gget.pdb(...)` — fetch a PDB entry.
- `gget.elm(...)` — short linear motif search.
- `gget.bgee(...)` — orthologs + expression atlases.
- `gget.cellxgene(...)` — query the CZ CELLxGENE single-cell atlas.
- `gget.cosmic(...)` — somatic mutations from COSMIC.
- `gget.muscle(...)` — multiple sequence alignment.

- `gget.blast(...)` — submit a BLAST job to NCBI, poll until done.
- `gget.blat(...)` — UCSC BLAT for short queries on a genome.
- `gget.diamond(...)` — local DIAMOND wrapper.

Every call returns a DataFrame — the plotting layer is the same matplotlib you already know.

gget.search • Find a Gene by Name

GGET • LOOKUP

From free text to Ensembl IDs

The simplest gget call: hand it a search term and a species, get back a DataFrame of matching Ensembl genes. Below is the call **and** the cached snapshot of what it returned on 2026-06-22 — paste the call into Colab to refresh, and check the output looks like the table.

```
1 import gget                                     # network-backed lookup
2
3 # eval: false on this chunk — needs network + a live Ensembl REST endpoint
4 hits = gget.search(searchwords=["brca1"],        # free text, list allow
5                     species="homo_sapiens",
6                     id_type="gene")             # gene | transcript
7 hits.head()                                     # returns a tidy DataFr
8 # columns: ensembl_id, gene_name, ensembl_description,
9 #           biotype, url
```

	ensembl_id	gene_name	biotype	ensembl
0	ENSG00000012048	BRCA1	protein_coding	BRCA1 [Source: Symbo...
1	ENSG00000267595	BRCA1P1	lncRNA	BRCA1 p1 [Source: Symbol;
2	ENSG00000225914	RPL21P4	processed_pseudogene	ribosom L21 pseu (near BF

gget.enrichr • Pathways for a Gene List

GGET • ENRICHMENT

Five cancer genes → top KEGG pathways → ranked bar chart

Pathway enrichment is the prototypical “many genes → one figure” answer. `gget.enrichr` posts a gene list to **Enrichr** and returns one row per pathway with the adjusted p-value, the overlapping genes, and the rank. Below: the **call** (network-gated, `eval: false`), a **cached snapshot** of the top hits embedded literally, then the bar plot of `-log10(adj_pval)` you build from it.

```
1 import gget
2
3 # --- LIVE call (eval: false; network required) -----
4 genes = ["BRCA1", "TP53", "EGFR", "MYC", "KRAS"] # five cancer drivers
5 enr = gget.enrichr(genes,
6                   database="KEGG_2021_Human") # one of ~200 libraries
7 enr.head(10) # columns below
8 # columns: rank, path_name, p_val, z_score, combined_score,
9 #          overlapping_genes, adj_p_val, database
10
11 # --- CACHED snapshot: top 10 KEGG_2021_Human hits, 2026-06-22 -----
12 cached = pd.DataFrame([
13     ("Pathways in cancer", 1, 1.2e-6, 1.4e-5, ["BI
14     ("Endometrial cancer", 2, 4.3e-6, 3.1e-5, ["TI
15     ("Non-small cell lung cancer", 3, 7.1e-6, 4.0e-5, ["TI
16     ("Bladder cancer", 4, 9.8e-6, 4.8e-5, ["TI
17     ("Colorectal cancer", 5, 1.5e-5, 6.6e-5, ["TI
18     ("p53 signaling pathway", 6, 2.0e-5, 7.7e-5, ["BI
19     ("MAPK signaling pathway", 7, 3.4e-5, 1.1e-4, ["EC
20     ("Pancreatic cancer", 8, 4.9e-5, 1.4e-4, ["TI
21     ("Glioma", 9, 6.7e-5, 1.7e-4, ["TI
22     ("ErbB signaling pathway", 10, 9.2e-5, 2.2e-4, ["EC
```

gget • Takeaway

GGET • RECAP

A live API behind a pandas-shaped door

THE RECIPE

1. **gget is the Swiss Army knife** for live gene / pathway / protein lookups — Ensembl, UniProt, BLAST, Enrichr, AlphaFold, CELLxGENE, COSMIC. 2. **Reach for it when the question is “what is this gene / what pathway is it in”** — not when the question is “plot my data”. The plotting stays matplotlib. 3. **Cache the DataFrame to disk** alongside the query date. APIs evolve; a paper figure must be reproducible against a snapshot, not a live endpoint. 4. **Network required.** Pin a snapshot for any rendered figure; the live call belongs in a notebook the reader can re-execute.

Compare with: the volcano + heatmap pipelines from day-1 — those answer “plot the differential expression I already computed”. `gget` is the layer **upstream** of that: it tells you *what genes are interesting* and *what pathways they sit in* so you can pick the right rows to plot. The two halves stitch together: `gget.enrichr` chooses the pathway, your own DESeq2 table colours the volcano.

Biopython • Phylo & Graphics

TREE IO + FEATURE DIAGRAMS

Biopython • Install + Theory

BIOPYTHON • SETUP

The plumbing: read every format, draw on matplotlib

```
1 pip install biopython
```

```
1 from io import StringIO          # read newick from a string
2 from Bio import Phylo, SeqIO     # tree IO + record IO
3 from Bio.Graphics import GenomeDiagram # circular + linear maps
4 from Bio.SeqRecord import SeqRecord # holds a sequence + features
5 from Bio.Seq import Seq          # the sequence itself
6 from Bio.SeqFeature import SeqFeature, SimpleLocation # gene / CDS features
```

Theory. Biopython is the **plumbing of bioinformatics**: it reads/writes **every common format** (Newick, Nexus, PhyloXML, FASTA, GenBank, EMBL ...) and exposes the result as plain Python objects.

- `Bio.Phylo` — tree IO + a matplotlib `draw()` that lays out the tree on a normal `Axes`. Stack it above a heatmap, side-by-side with a Manhattan plot, anywhere you'd put any other matplotlib panel.
- `Bio.Graphics.GenomeDiagram` — high-level builder for **publication-quality plasmid maps** (circular) and **linear gene tracks** with arrow sigils. Renders to PNG/SVG/PDF directly; no matplotlib needed under the hood.

Biopython • The Data

BIOPYTHON • DATA PEEK

Same 9-tip Newick, parsed by Phylo + ASCII preview

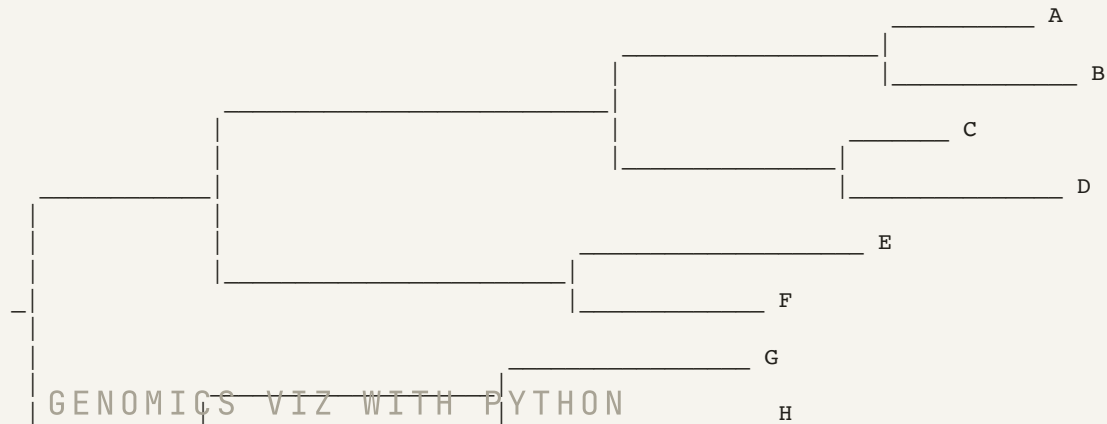
We reuse the Newick string from the Toytree section.

`Phylo.read(StringIO(...))` returns a `Tree` object;

`Phylo.draw_ascii` prints it as text — useful for log files
and sanity-checking parsing before drawing.

```
1 from io import StringIO          # treat the string as a file
2 from Bio import Phylo           # tree IO + draw
3
4 # same 9-tip phylogeny we used in the Toytree section
5 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0, \" \
6         \"(E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);\"
7
8 tree = Phylo.read(StringIO(newick), "newick")    # parse Newick -> Tree
9 print("terminals:", [t.name for t in tree.get_terminals()]) # leaf names
10 Phylo.draw_ascii(tree)                         # text-art preview
```

terminals: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']



Biopython · Step 1 — `Phylo.draw` on Matplotlib

BIOPYTHON · BUILD

1 · Draws on the current axes — `None` return value

`Phylo.draw(tree)` does **not** return a figure; it draws onto the **current matplotlib axes** (or creates one).

Create the `Axes` yourself with `plt.subplots()` so you control the size + can stack the tree with other panels.

```
1 from io import StringIO          # newick from string
2 from Bio import Phylo           # tree IO + draw
3
4 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0, \
5         \"(E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);\"
6 tree = Phylo.read(StringIO(newick), \"newick\")    # parse once
7
8 fig, ax = plt.subplots(figsize=(6.5, 4.5))        # CREATE axes for th
9 Phylo.draw(                                       # CALL — draws on `a
10     tree,
11     axes=ax,                                     # explicit axes targ
12     do_show=False,                               # don't pop a window
13 )
14 ax.spines[["top", "right"]].set_visible(False)   # match cream style
```

Biopython · Step 2 — Colour Branches by Clade

BIOPYTHON · BUILD

2 · Walk the tree, set `clade.color`

Bio.Phylo's `Tree` is a recursive structure. Walk it with `tree.find_clades()`, set `clade.color = "#hex"` on each non-terminal subtree, and `Phylo.draw` picks up the colours automatically. Below: each top-level clade in a different Economist hue.

```
1 from io import StringIO                                # newick from string
2 from Bio import Phylo                                  # tree IO + draw
3
4 newick = "(((A:1.2,B:1.5):2.1,(C:0.9,D:1.7):1.8):3.0, \
5          \"(E:2.3,F:1.6):2.7):1.5,((G:1.9,H:2.0):2.3,I:3.5):1.4);"
6 tree = Phylo.read(StringIO(newick), "newick")          # parse Tree
7
8 # colour the two top-level children of the root in two course hues
9 palette = [GREEN, BLUE]                                # one hue per major
10 for clade, colour in zip(tree.root.clades, palette):   # iterate top-level
11     clade.color = colour                                # set hex; Phylo.dra
12
13 # CREATE the axes, CALL Phylo.draw — colours come through automatically
14 fig, ax = plt.subplots(figsize=(6.5, 4.5))            # CREATE axes
15 Phylo.draw(tree, axes=ax, do_show=False)              # CALL — picks up cl
16 ax.spines[["top", "right"]].set_visible(False)        # cream style
```


Biopython · Step 3 — Circular Plasmid Diagram

BIOPYTHON · BUILD

3 · `GenomeDiagram` — five genes on a 5 kb plasmid

`GenomeDiagram` builds publication-quality plasmid maps. The pattern: 1. wrap the sequence in a `SeqRecord`, 2. attach `SeqFeature` objects (gene boxes), 3. create a `Diagram`, add a `Track`, add a `FeatureSet`, 4. `.draw(format="circular", ...)`, 5. `.write(path, "PNG")`.

We embed the resulting PNG with ``.

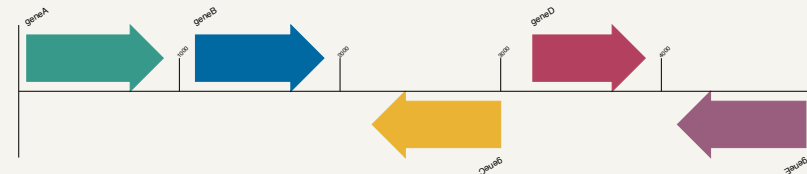
Biopython · Step 4 — Linear Gene Track

BIOPYTHON · BUILD

4 · Same features, `format="linear"`, ARROW sigils

Switch `format="circular"` to `"linear"` and the same `Diagram` redraws as a horizontal gene track. Arrows now point in the strand direction — forward genes right, reverse genes left. Same data, same colours, different geometry.

```
1 from pathlib import Path                                # cross-platform
2 from Bio.SeqRecord import SeqRecord; from Bio.Seq import Seq
3 from Bio.SeqFeature import SeqFeature, SimpleLocation
4 from Bio.Graphics import GenomeDiagram                 # the diagram bui
5
6 # same 5 gene features as the circular step
7 record = SeqRecord(Seq("N" * 5000), id="plasmid")
8 gene_spans = [( 50, 900, +1, "geneA", GREEN),
9               (1100, 1900, +1, "geneB", BLUE),
10              (2200, 3000, -1, "geneC", AMBER),
11              (3200, 3900, +1, "geneD", RED),
12              (4100, 4900, -1, "geneE", PURPLE)]
13 for s, e, strand, name, _ in gene_spans:
14     record.features.append(SeqFeature(
15         SimpleLocation(s, e, strand=strand),
16         type="gene", qualifiers={"label": [name]}))
17
18 gd = GenomeDiagram.Diagram("toy genome")               # new top-level c
19 trk = gd.new_track(1, name="genes", greytrack=False,   # one horizontal
20                    scale_targetick_interval=1000)
21 fs = trk.new_set()
22 for (s, e, strand, name, hue) in zip(gene_spans, record.features):
```



Biopython • Takeaway

BIOPYTHON • RECAP

The plumbing for trees + feature diagrams

THE RECIPE

1. `Phylo.read(file, format)` parses Newick / Nexus / PhyloXML; `Phylo.draw` lays the tree on any matplotlib `Axes`. 2. `clade.color = "#hex"` colours subtrees in-place — `Phylo.draw` picks it up. 3. `GenomeDiagram` stacks `Diagram → Track → FeatureSet`; one `Diagram` can render circular **and** linear without rebuilding the features. 4. `sigil="ARROW"` + strand sign gives canonical gene-track arrows. 5. Output is **PNG / SVG / PDF directly** — no matplotlib needed under the hood.

Compare with: pair `Phylo` with **Toytree** (section 3) —

Phylo for IO, Toytree for prettier display. Pair

`Bio.Graphics` with `dna_features_viewer` (section 7)

— Bio.Graphics for plasmid-style circular publication output, `dna_features_viewer` for matplotlib-native gene maps you can stack with the rest of your figure.

Biopython is the plumbing; the other libraries are the polish.